

Computer Vision 1

Global Transforms & Image Stitching

Dr. Martin R. Oswald

My Background



- **Studies of Computer Science and Engineering**
Technical University of Dresden, Germany



University of Technology Sydney, Australia



Universidad Técnica Federico Santa María, Valpariso, Chile

- **PhD**

University of Bonn, Germany



Technical University of Munich, Germany



- **Senior Researcher**
Eidgenössische Technische Hochschule Zürich, Switzerland
- **Assistant Professor**
University of Amsterdam, Netherlands

ETH zürich



European Conference on Computer Vision

Sep 29 – Oct 4, 2024 – Milano, Italy



- Lectures and tutorials will be hybrid/online for Sep 26 - Oct 3

Lecture overview

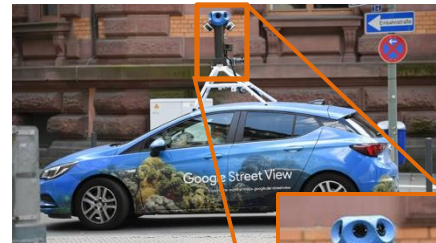
Week		Date	Presenter	Lectures	Exercise / Labs
1	36	02/09/2024	Arnoud Visser	1. Introduction and Course Logistics; Camera model	
1	36	05/09/2024	Arnoud Visser	2. Sensor and image formation Color	
2	37	09/09/2024	Arnoud Visser	3. In place image processing Photometric stereo	
2	37	12/09/2024	Arnoud Visser	4. Convolution and image filtering	
3	38	16/09/2024	Arnoud Visser	5. Feature: edge and corner	
3	38	19/09/2024	Arnoud Visser	6. Motion and optical flow	
4	39	23/09/2024	Martin Oswald	7. Global transform, Image stitching, alignment	
4	39	26/09/2024	Martin Oswald	8. Basics of Recognition 1	
5	40	30/09/2024	Martin Oswald	9. Basics of Recognition 2	
5	40	03/10/2024	Martin Oswald	10. Convolutional neural network 1 (Basic)	
6	41	07/10/2024	Martin Oswald	11. Convolutional neural network (Training and testing)	
6	41	10/10/2024	Martin Oswald	12. Stereo, 3D reconstruction	
7	42	14/10/2024	TBA	13. Guest lectures	
7	42	17/10/2024	TBA; Martin & Arnoud	14. Guest lecture; Course Wrap-up, Exam Q&A	
8	43	21/10/2024	Your turn!	Final exam	

Today

Online
Lectures

Today's Motivation: How to do Panoramas?

360 photo sphere panorama
feature on pixel 6.



**Google
Street View**



= Huge collection of stitched
spherical images

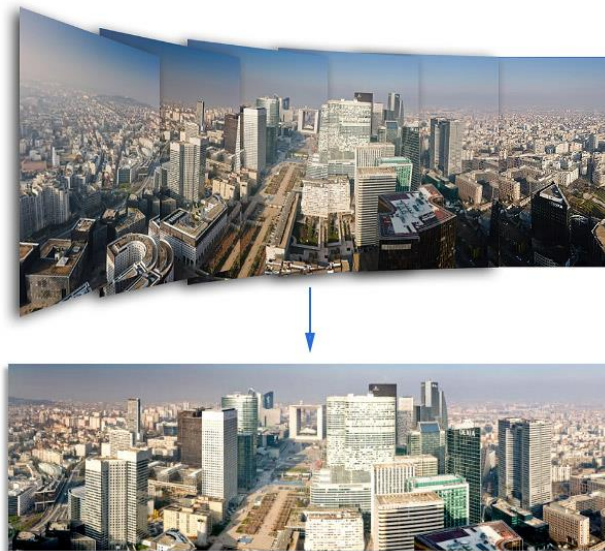
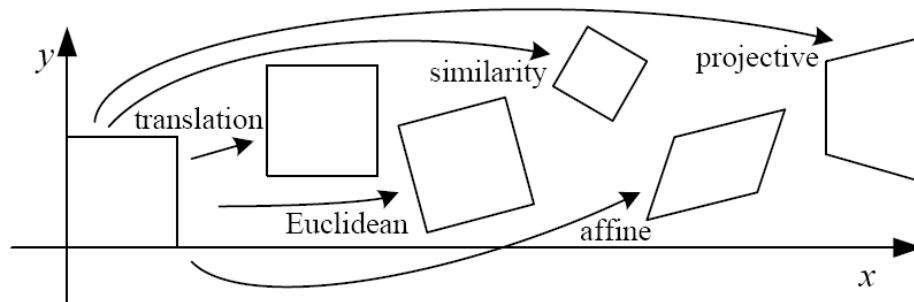
Today's Agenda

- **Global operations**

- Global transformations
- Image Warping
- Image Morphing

- **Image Stitching**

- Transformation computation
 - Reprojection model
 - Key point matching
 - RANSAC
 - Solving transformation
- Planar mapping
- Blending
 - Feathering
 - Multi-band blending
 - Further improvements



- Chapter 8

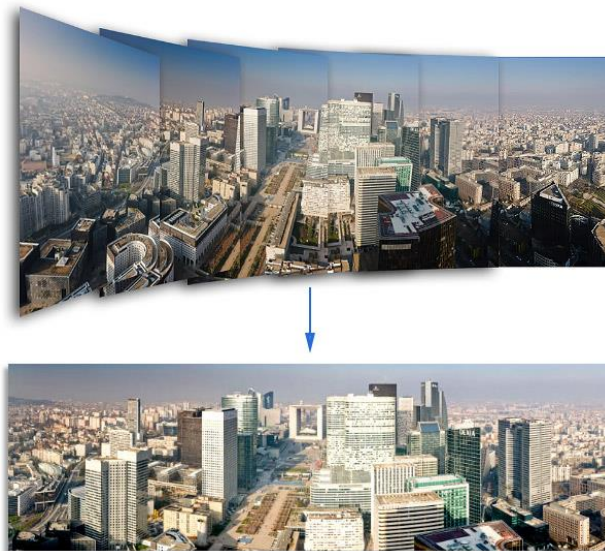
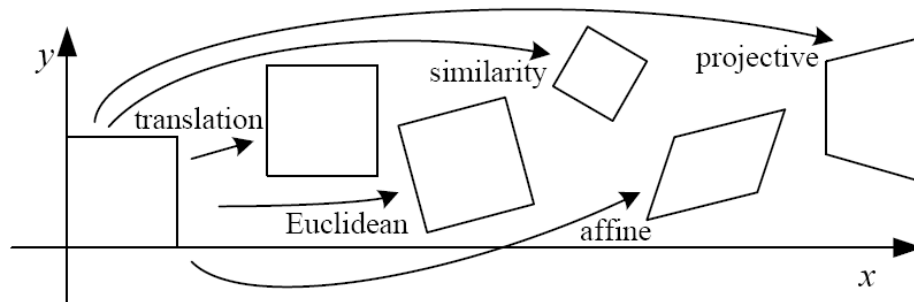
Today's Agenda

- **Global operations**

- Global transformations
- Image Warping
- Image Morphing

- **Image Stitching**

- Transformation computation
 - Reprojection model
 - Key point matching
 - Solving transformation
 - RANSAC
- Planar mapping
- Blending
 - Feathering
 - Multi-band blending
 - Further improvements



Parametric (Global) Transformations



$$\mathbf{p} = (x, y)$$



$$\mathbf{p}' = (x', y')$$

Transformation T is a coordinate-changing operation:

$$\mathbf{p}' = T(\mathbf{p})$$

What does it mean that T is global?

- T is the same for any point p in the entire image domain

T can be described by just a few numbers (parameters)

For linear transformations, we can represent T as a matrix

$$\mathbf{p}' = \mathbf{T}\mathbf{p}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \mathbf{T} \begin{bmatrix} x \\ y \end{bmatrix}$$

Common Linear Transformations



Original

Transformed



Translation



Rotation



Scaling



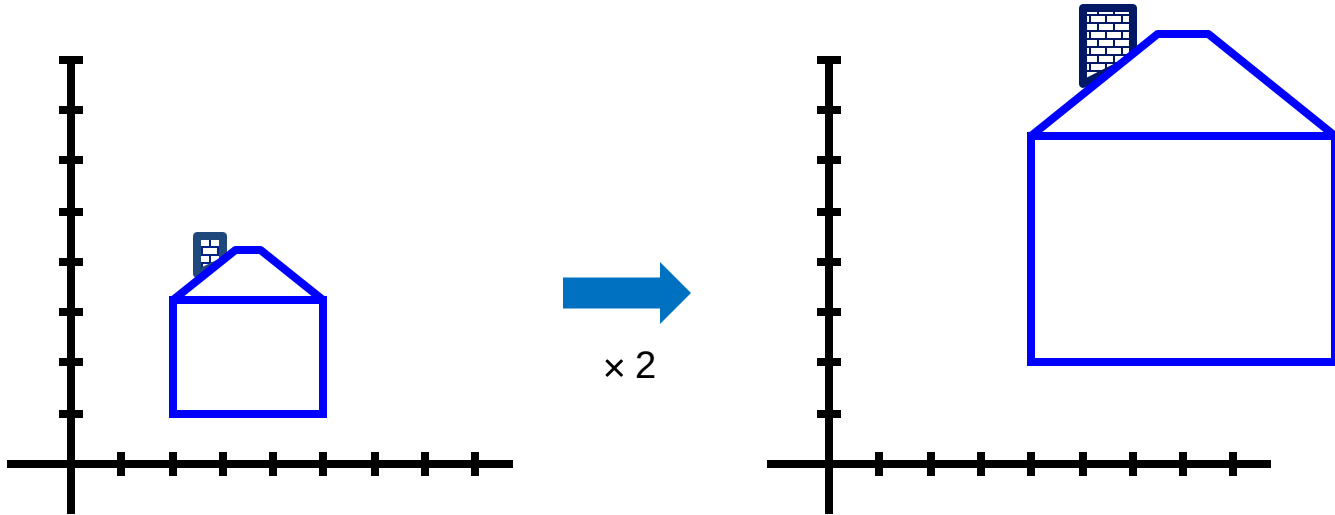
Affine



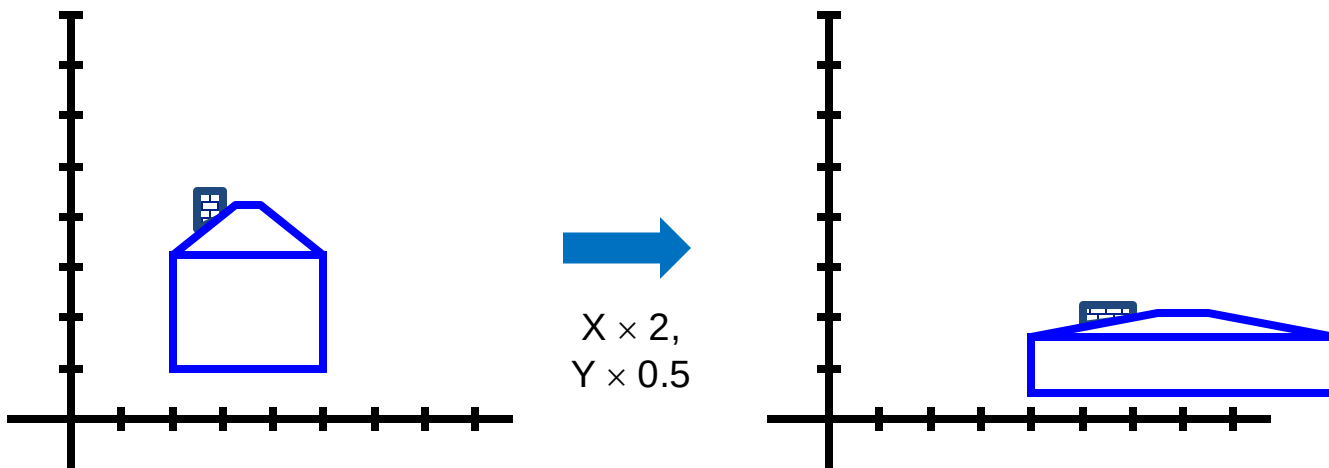
Perspective

Scaling

- *Scaling* a coordinate means multiplying each of its components by a scalar
- *Uniform scaling* means this scalar is the same for all components:



- *Non-uniform scaling*: different scalars per component:



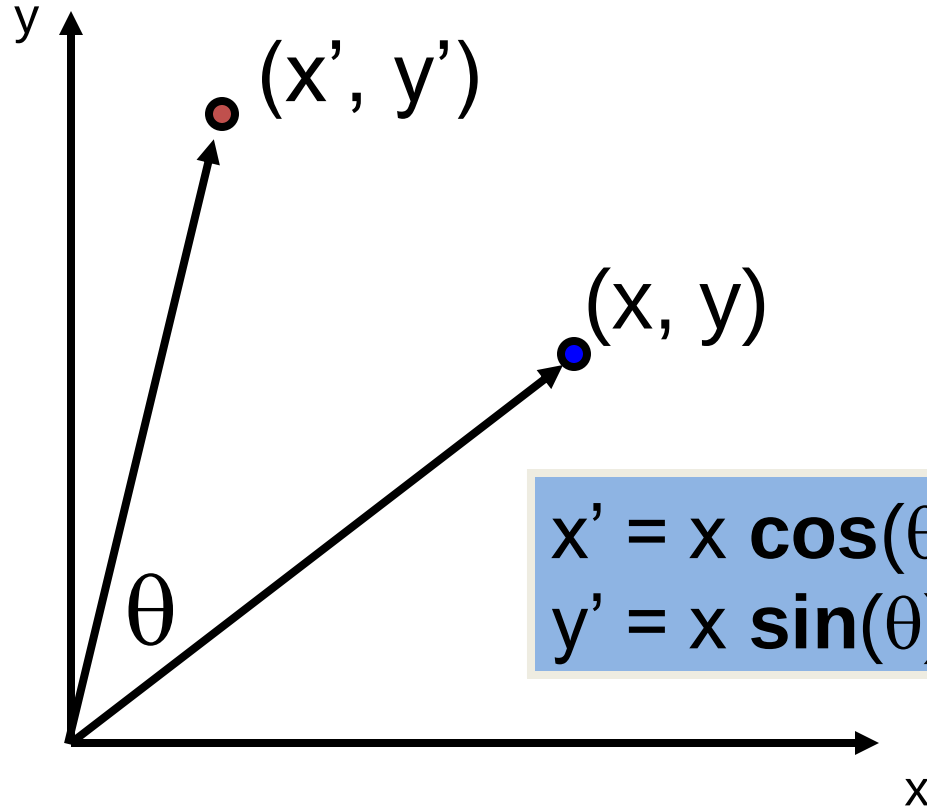
Scaling

- Scaling operation: $x' = ax$
 $y' = by$

- Or, in matrix form:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \underbrace{\begin{bmatrix} a & 0 \\ 0 & b \end{bmatrix}}_{\text{scaling matrix } S} \begin{bmatrix} x \\ y \end{bmatrix}$$

2D Rotation



$$\begin{aligned}x' &= x \cos(\theta) - y \sin(\theta) \\y' &= x \sin(\theta) + y \cos(\theta)\end{aligned}$$

2D Rotation

This is easy to capture in matrix form:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \underbrace{\begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}}_{\mathbf{R}} \begin{bmatrix} x \\ y \end{bmatrix}$$

Even though $\sin(\theta)$ and $\cos(\theta)$ are nonlinear functions of θ ,

- x' is a linear combination of x and y
- y' is a linear combination of x and y

What is the inverse transformation?

- Rotation by $-\theta$
- For rotation matrices $\mathbf{R}^{-1} = \mathbf{R}^T$ (because of orthogonality)

- Q: How can we represent translation as a 3x3 matrix?

$$\mathbf{x}' = \mathbf{x} + \mathbf{t}_x$$

$$\mathbf{y}' = \mathbf{y} + \mathbf{t}_y$$

2D Translation

- *Homogeneous coordinates*
→ represent coordinates in
2 dimensions with a 3-vector

$$\begin{bmatrix} x \\ y \end{bmatrix} \xrightarrow{\text{homogeneous coords}} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

2D Translation

- Using the rightmost column:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} x + t_x \\ y + t_y \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

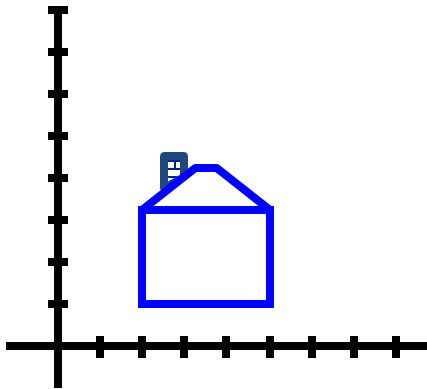
2D Translation

- Example of translation

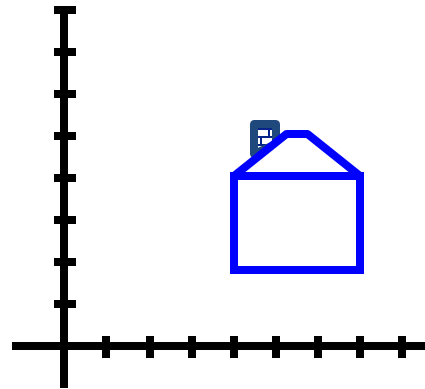
Homogeneous Coordinates



$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + t_x \\ y + t_y \\ 1 \end{bmatrix}$$



$$\begin{aligned} t_x &= 2 \\ t_y &= 1 \end{aligned}$$



Basic 2D Transformations

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Translate

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Scale

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \Theta & -\sin \Theta & 0 \\ \sin \Theta & \cos \Theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Rotate

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & sh_x & 0 \\ sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Shear

Affine Transformations

- Affine transformations are combinations of ...
 - Linear transformations, and
 - Translations
- Properties of affine transformations:
 - Origin does not necessarily map to origin
 - Lines map to lines
 - Parallel lines remain parallel
 - Ratios are preserved
 - Closed under composition
 - Models change of basis

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} d_x \\ d_y \end{bmatrix}$$



$$\begin{bmatrix} x' \\ y' \\ w \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

Basic 2D Transformations

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Scale

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & \alpha_x \\ \alpha_y & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Shear

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \Theta & -\sin \Theta \\ \sin \Theta & \cos \Theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Rotation

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Translation

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Affine

Affine is any combination of translation, scale, rotation, and shear

Transformation Composition

- Transformations can be combined by matrix multiplication

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \left(\begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \Theta & -\sin \Theta & 0 \\ \sin \Theta & \cos \Theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{bmatrix} \right) \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

$$\mathbf{p}' = \mathbf{T}(t_x, t_y) \mathbf{R}(\Theta) \mathbf{S}(s_x, s_y) \mathbf{p}$$

Projective Transformations (a.k.a. Homographies)

- Projective transformations ...

- Affine transformations, and
- Projective warps

$$x' = \frac{a_1x + a_2y + a_3}{a_7x + a_8y + 1}$$

- Properties of projective transformations:

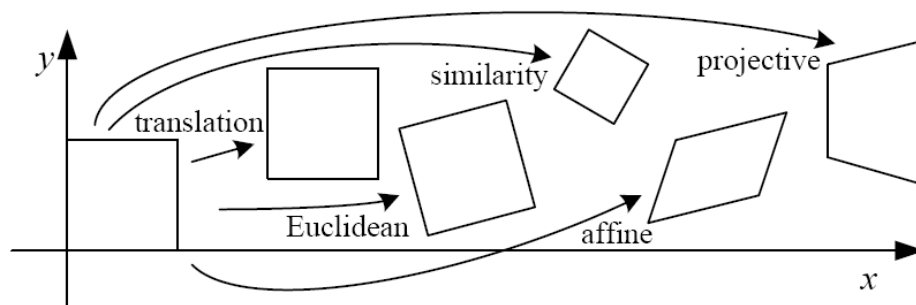
- Origin does not necessarily map to origin
- Lines map to lines
- Parallel lines do not necessarily remain parallel
- Ratios are not preserved
- Closed under composition
- Models change of basis

$$y' = \frac{a_4x + a_5y + a_6}{a_7x + a_8y + 1}$$



$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

Basic 2D Transformations



Name	Matrix	# D.O.F.	Preserves:	Icon
translation	$\begin{bmatrix} I & t \end{bmatrix}_{2 \times 3}$	2	orientation + ...	
rigid (Euclidean)	$\begin{bmatrix} R & t \end{bmatrix}_{2 \times 3}$	3	lengths + ...	
similarity	$\begin{bmatrix} sR & t \end{bmatrix}_{2 \times 3}$	4	angles + ...	
affine	$\begin{bmatrix} A \end{bmatrix}_{2 \times 3}$	6	parallelism + ...	
projective	$\begin{bmatrix} \tilde{H} \end{bmatrix}_{3 \times 3}$	8	straight lines	

These transformations are a nested set of groups.
Closed under composition and inverse is a member.

Reminder: Groups

A **group** is a set G together with a binary operation on G , denoted “ $\cdot$ ”, that combines any two elements a and b to form an element of G , denoted $a \cdot b$, such that the following requirements (group axioms) are satisfied:

- **Closedness:** For all a, b in G , one has $(a \cdot b)$ in G .
- **Associativity:** For all a, b, c in G , one has $(a \cdot b) \cdot c = a \cdot (b \cdot c)$
- **Identity element:** There exists an element e in G such that, for every a in G , one has $e \cdot a = a$ and $a \cdot e = a$.

Such an element is unique and called the *identity element* of the group.

- **Inverse element:** For each a in G , there exists an element b in G such that $a \cdot b = e$ and $b \cdot a = e$, where e is the identity element.
For each a , the element b is unique; it is called the inverse of a and denoted a^{-1} .

Overview: Matrix Groups

Set of square $n \times n$ matrices	General Linear Group	Orthogonal Group	Special Orthogonal Group
	$\mathcal{M}(n) \supset GL(n) = \{A \in \mathcal{M}(n) \mid \det(A) \neq 0\}$ Invertible square matrices	$O(n) = \{R \in GL(n) \mid R^T R = I\}$ Orthogonal matrices	$SO(n) = \{R \in O(n) \mid \det(R) = 1\}$ Rotation matrices

General Linear Group	Affine Group	Euclidean Group	Special Euclidean Group
	$GL(n+1) \supset \mathcal{A}(n) = \left\{ \begin{pmatrix} A & b \\ 0 & 1 \end{pmatrix} \mid A \in GL(n), b \in \mathbb{R}^n \right\}$ Affine matrices	$E(n) = \left\{ \begin{pmatrix} R & t \\ 0 & 1 \end{pmatrix} \mid R \in O(n), t \in \mathbb{R}^n \right\}$	$SE(n) = \left\{ \begin{pmatrix} R & t \\ 0 & 1 \end{pmatrix} \mid R \in SO(n), t \in \mathbb{R}^n \right\}$ Rigid-body motions

Today's Agenda

- **Global operations**

- Global transformations
- **Image Warping**
- Image Morphing

- **Image Stitching**

- Transformation computation
 - Reprojection model
 - Key point matching
 - Solving transformation
 - RANSAC
- Planar mapping
- Blending
 - Feathering
 - Multi-band blending
 - Further improvements

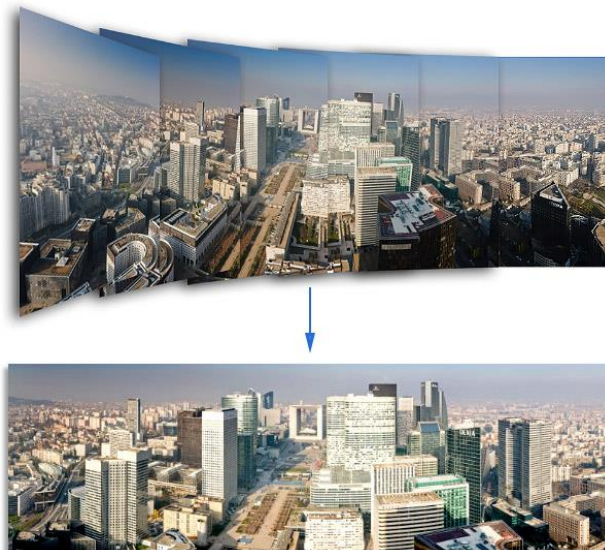
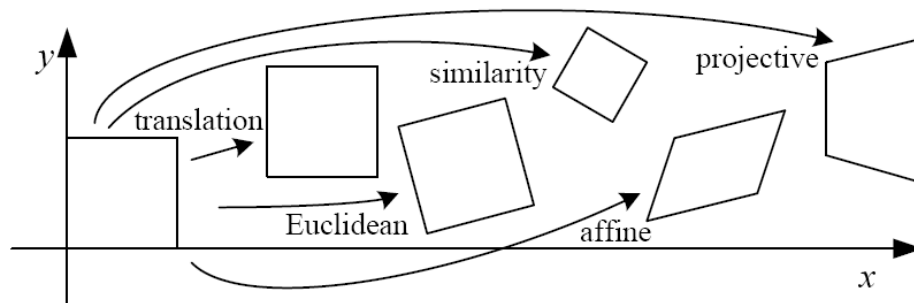
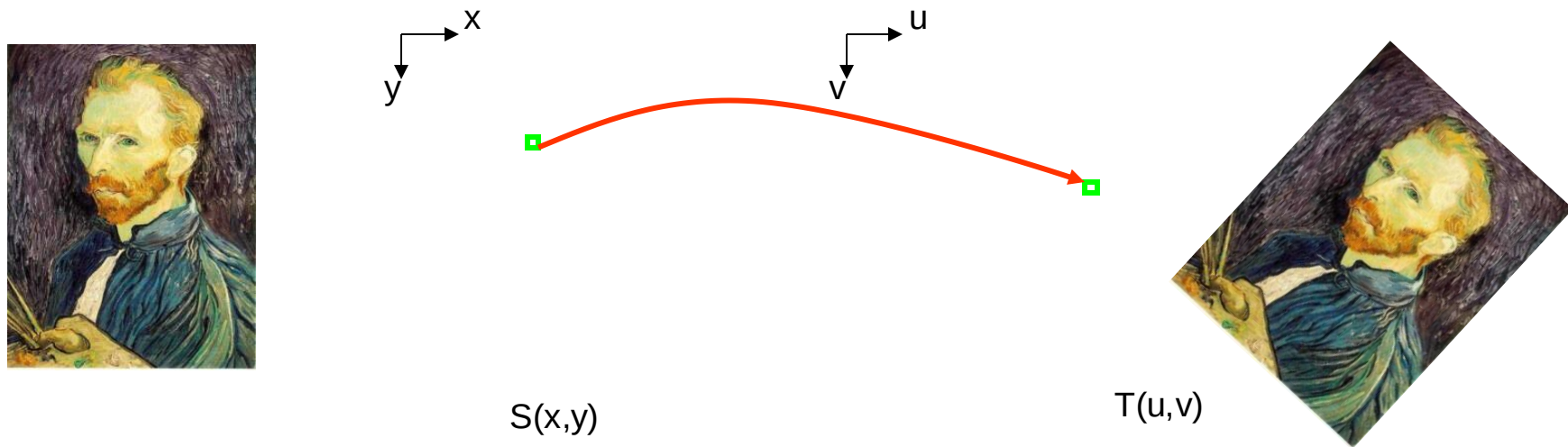


Image Warping



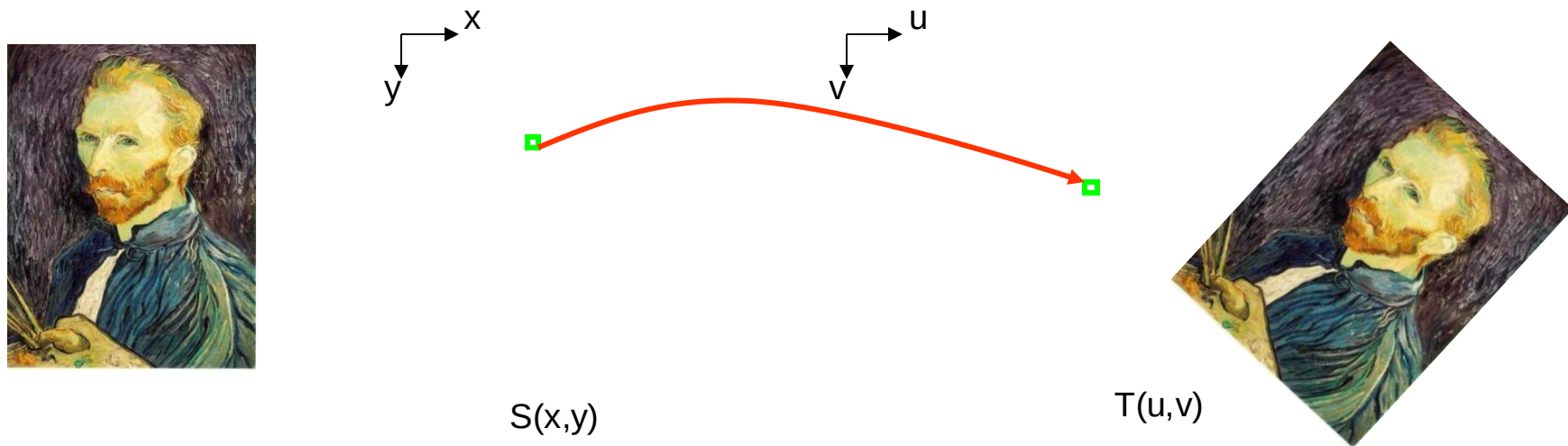
[Image source: <http://www.jeffrey-martin.com>]

Image Warping



Given a coordinate transform function $[f,g]$ (or the inverse i.e., $[F,G]$) and source image $S(x,y)$, how do we compute a transformed target image $T(u,v)$?

Forward Warping



Forward warping algorithm:

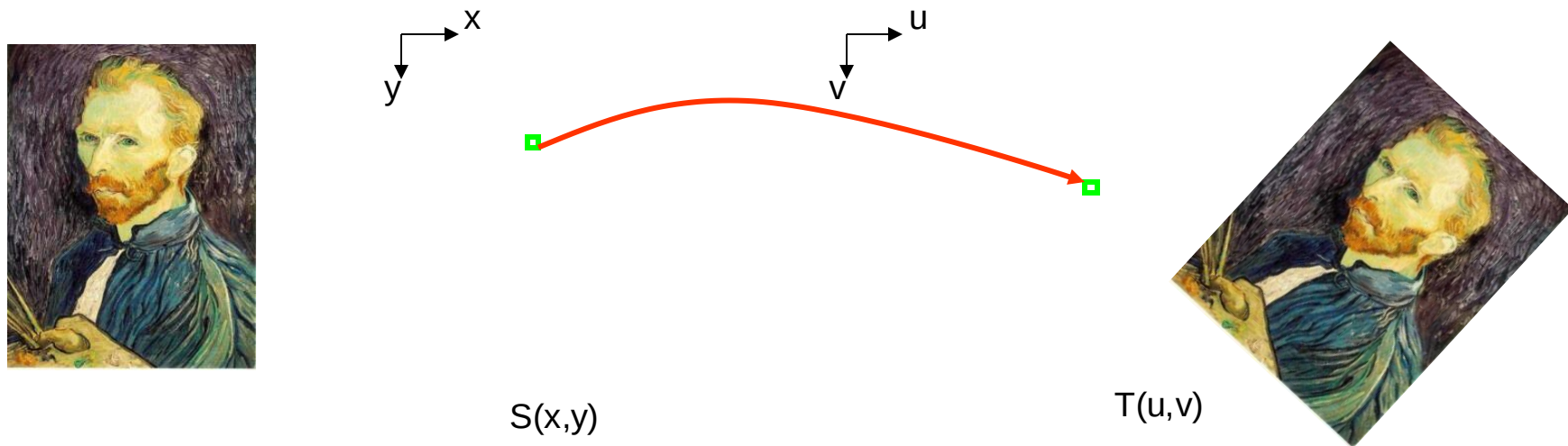
for $y = y_{\min}$ to $y_{\max}$

for $x = x_{\min}$ to $x_{\max}$

$u = f(x,y)$; $v = g(x,y)$

copy pixel at source $S(x,y)$ to $T(u,v)$

Forward Warping



Forward warping algorithm:

for $y = y_{\min}$ to $y_{\max}$

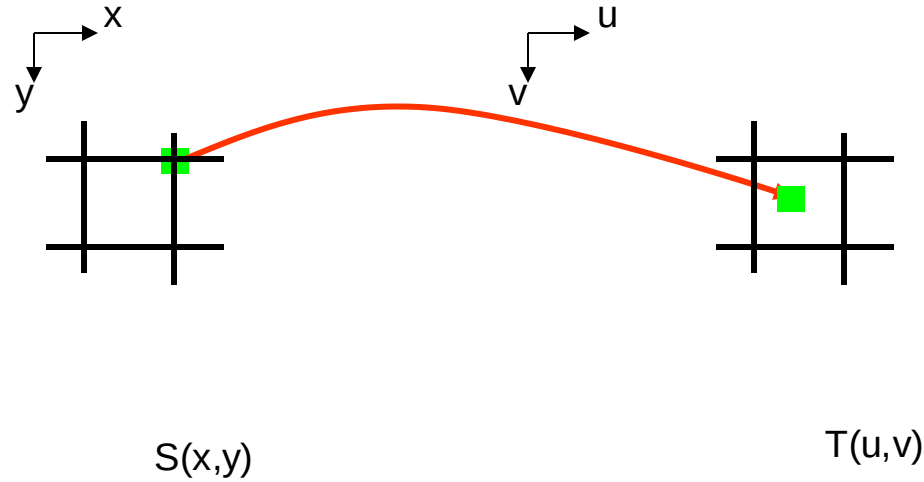
for $x = x_{\min}$ to $x_{\max}$

$u = f(x,y)$; $v = g(x,y)$

copy pixel at source $S(x,y)$ to $T(u,v)$

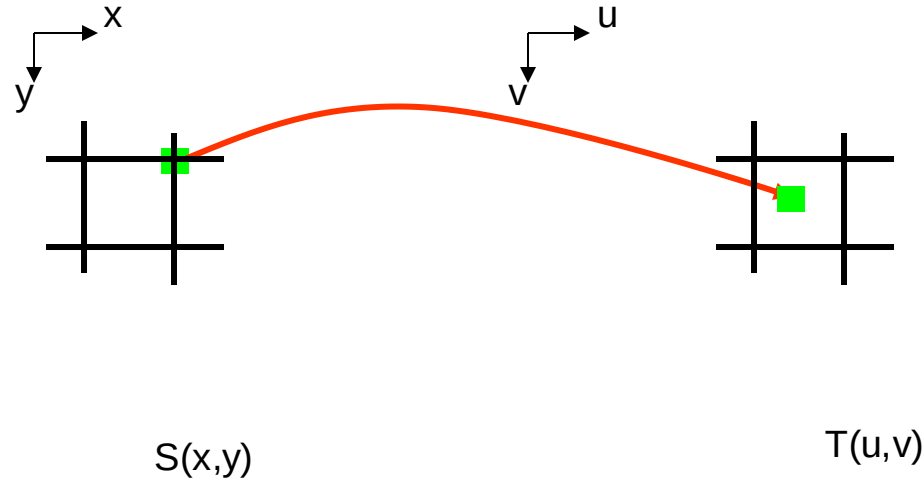
- Any problems for forward warping?

Forward Warping



Q: What if the transformed pixel located between pixels?

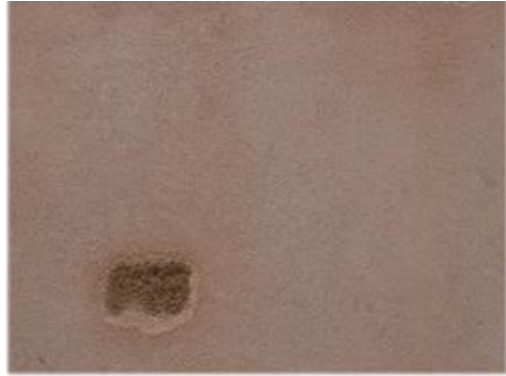
Forward Warping



Q: What if the transformed pixel located between pixels?

A: Distribute color among neighboring pixels
- known as “splatting”

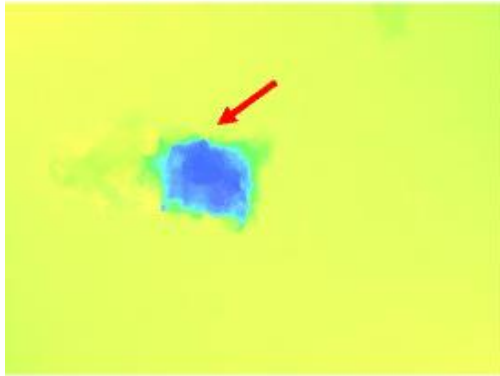
Forward Warping - Problems



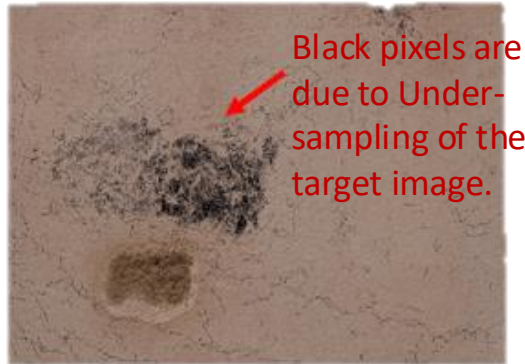
Target image



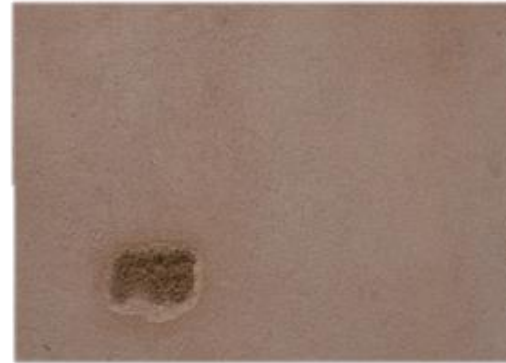
Reference image



Optical flow



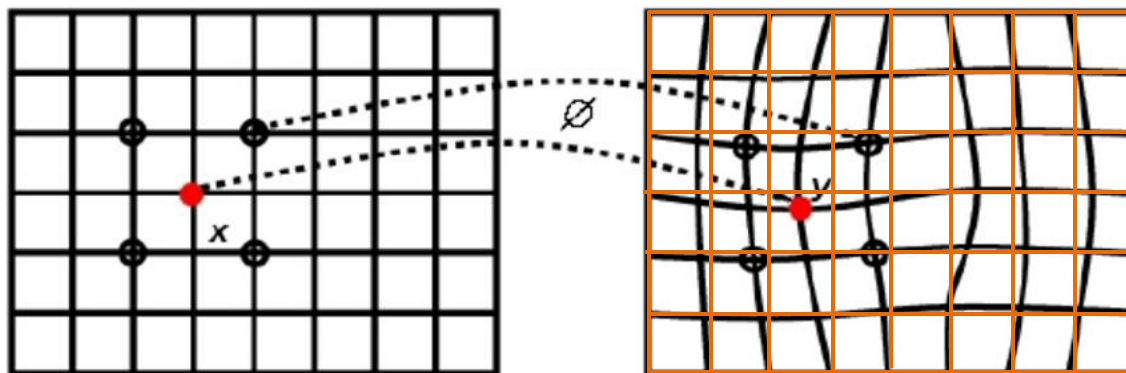
Forward warping



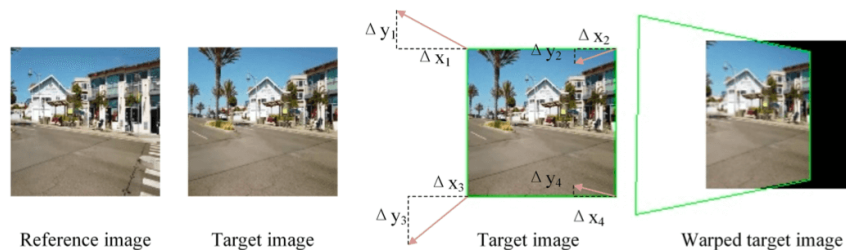
Backward warping

Forward Warping - Problems

- Target image is oversampled for converging flows
- Target image is undersampled for diverging flows

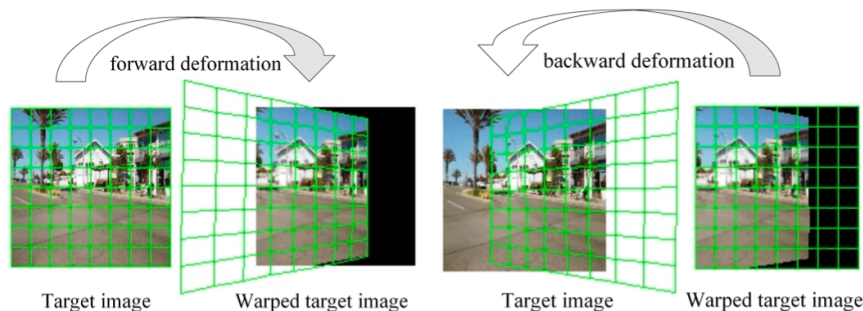


Forward vs. Backward Warping



(a) Input images.

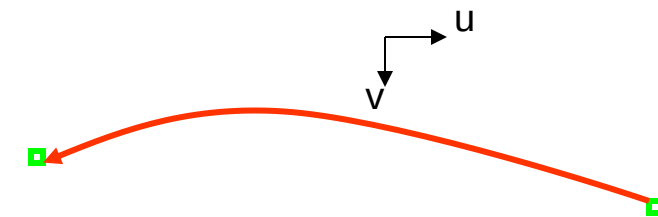
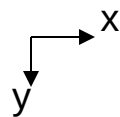
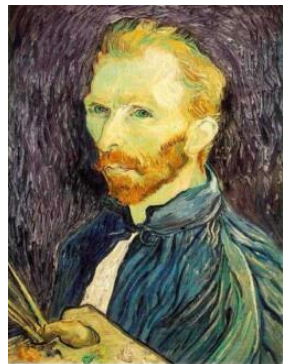
(b) Forward single grid deformation.



(c) Forward multi-grid deformation. (d) Backward multi-grid deformation.

Fig. 6. Comparison between the forward deformation and backward deformation.

Inverse/Backward Warping



$S(x,y)$



$T(u,v)$

Inverse warping algorithm:

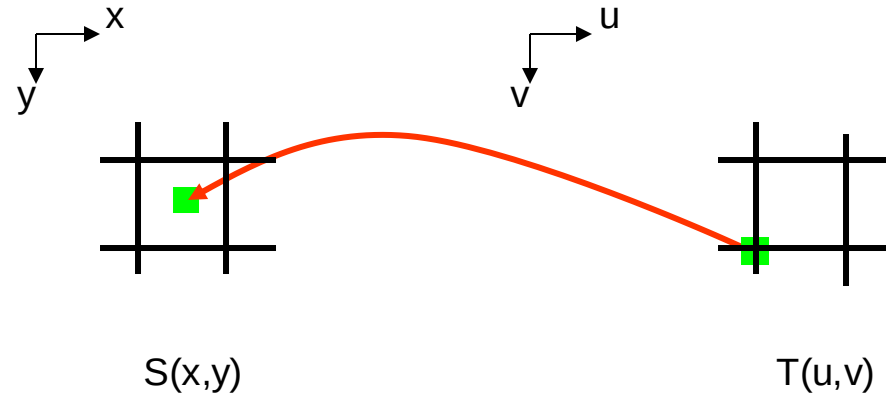
for $v = v_{\min}$ to $v_{\max}$

for $u = u_{\min}$ to $u_{\max}$

$x = F(u,v); y = G(u,v)$

copy pixel at source $S(x,y)$ to $T(u,v)$

Image Warping

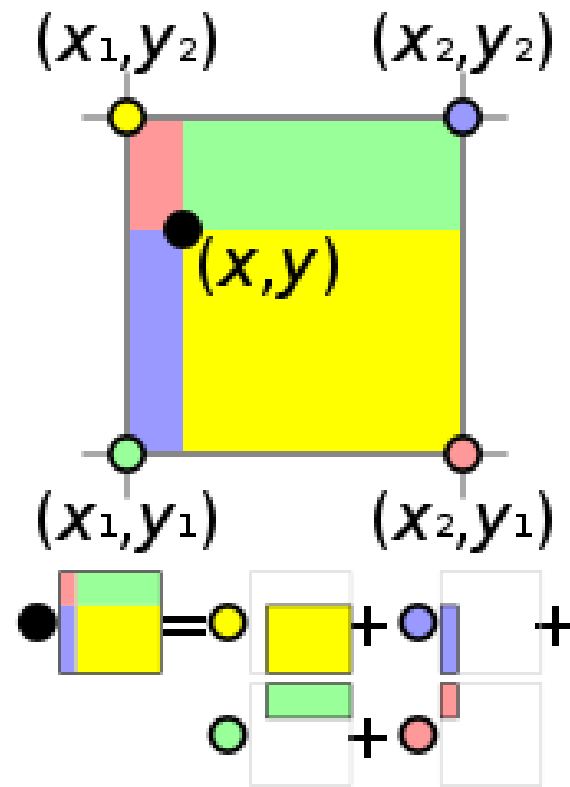


Q: What if pixel comes from “between” two pixels?

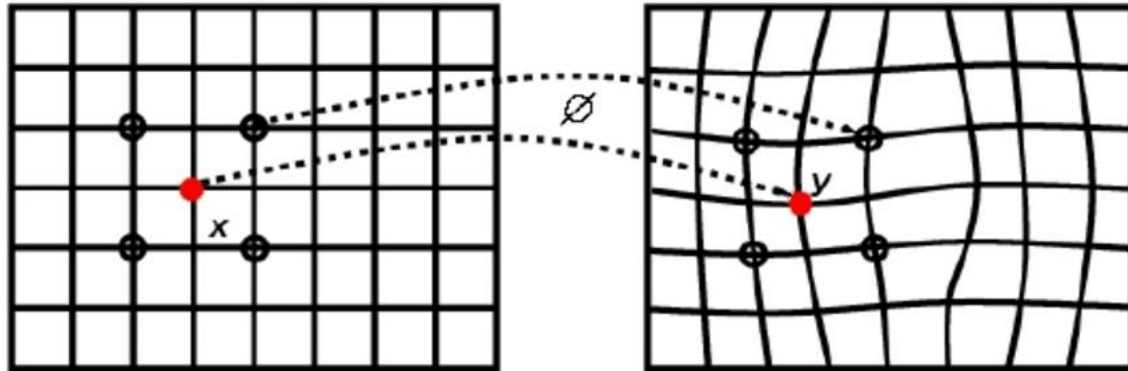
A: Interpolate color values from neighboring pixels

Bilinear Interpolation

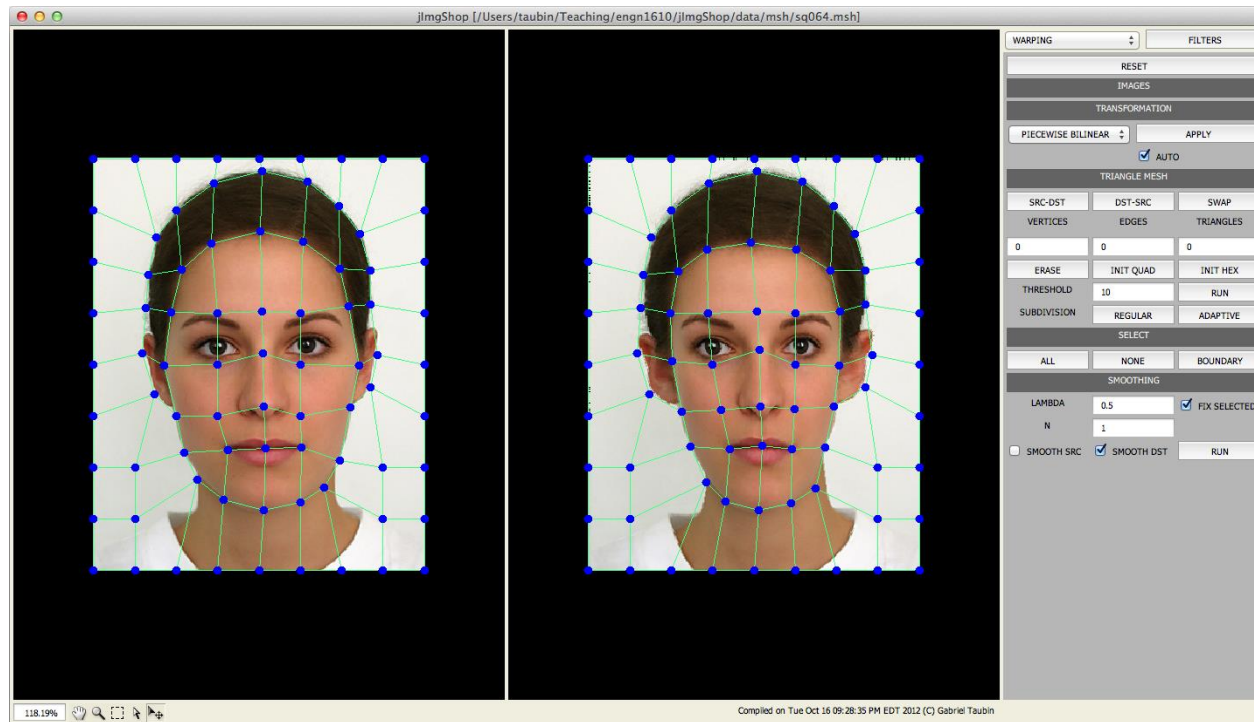
- **Goal:** Given an image with colors defined on integer grid locations, how to compute the color at a sub-pixel coordinate (x,y) ?
- **Approach:** The color of a point between discrete grid locations is defined as the weighted linear interpolation of the colors of the four pixel neighbors.
The weights are according to the area of the rectangle between point (x,y) and the corresponding neighbor.



Piece-wise Linear Warping



Piece-wise Linear Warping



Piece-wise Linear Warping

n point image deformation

Iterative ARAP
(has problems with stretching and shrinking)

Today's Agenda

- **Global operations**

- Global transformations
- Image Warping
- Image Morphing

- **Image Stitching**

- Transformation computation
 - Reprojection model
 - Key point matching
 - RANSAC
 - Solving transformation
- Planar mapping
- Blending
 - Feathering
 - Multi-band blending
 - Further improvements

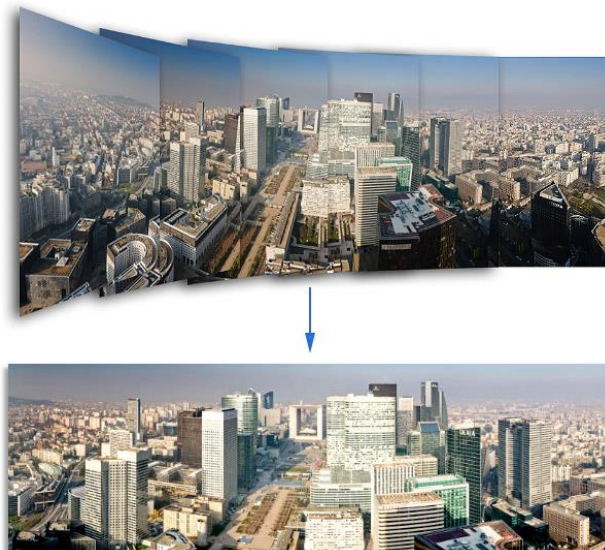
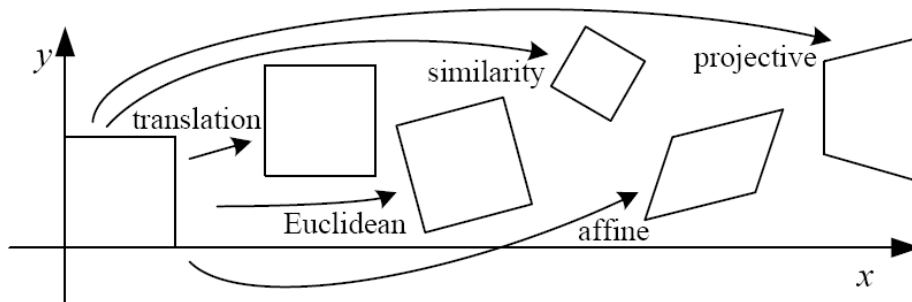


Image Morphing



- 1991 – First photorealistic face morphing
- Michael Jackson – Black or White (official music video)
- Lots of manual work: corresponding points for eyes, mouth, etc. were manually selected
- Back then: 4-5h of computing time for a few frames of a single morph transition



Image Morphing

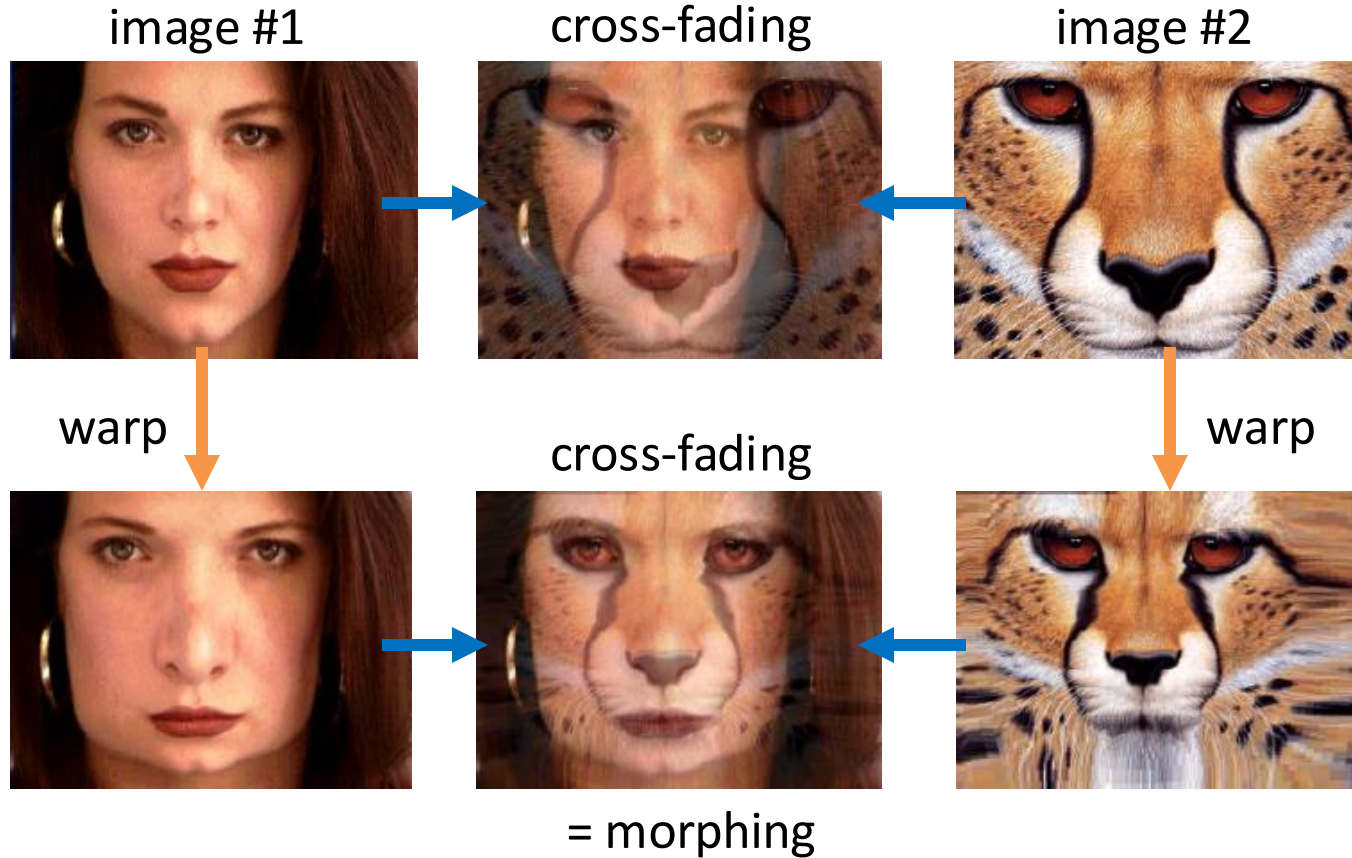


Image Morphing

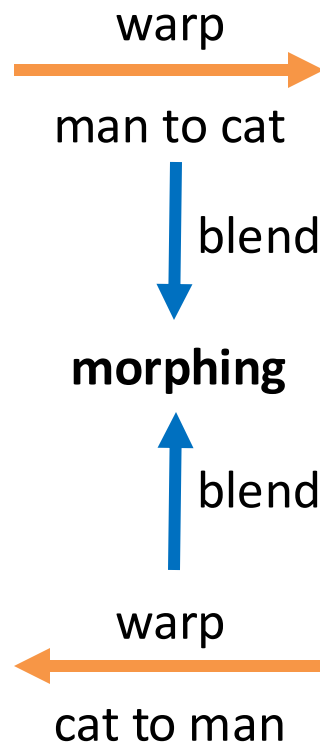
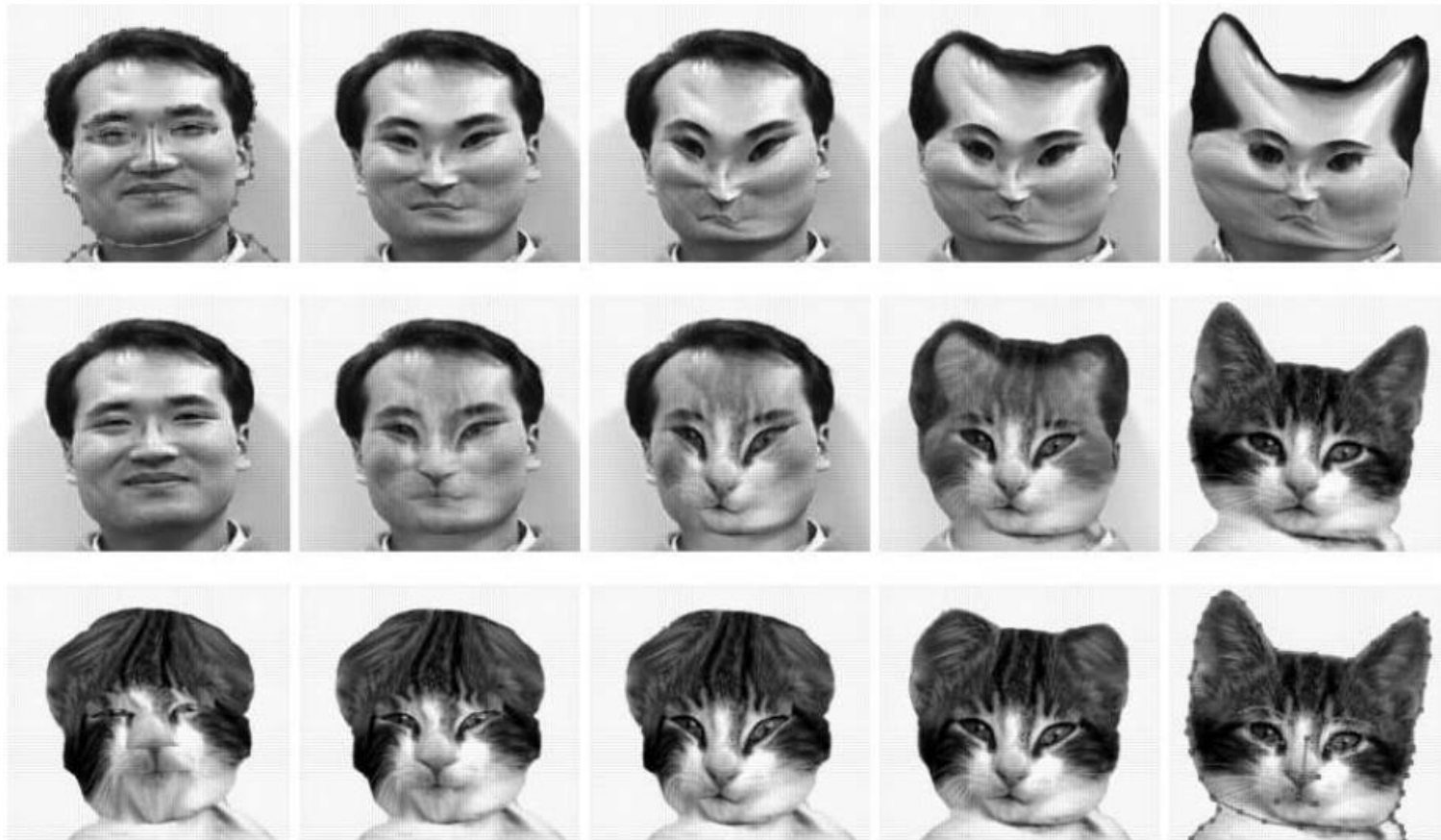


Image Morphing



Today's Agenda

- **Global operations**

- Global transformations
- Image Warping
- Image Morphing

- **Image Stitching**

- Transformation computation
 - Reprojection model
 - Key point matching
 - Solving transformation
 - RANSAC
- Planar mapping
- Blending
 - Feathering
 - Multi-band blending
 - Further improvements

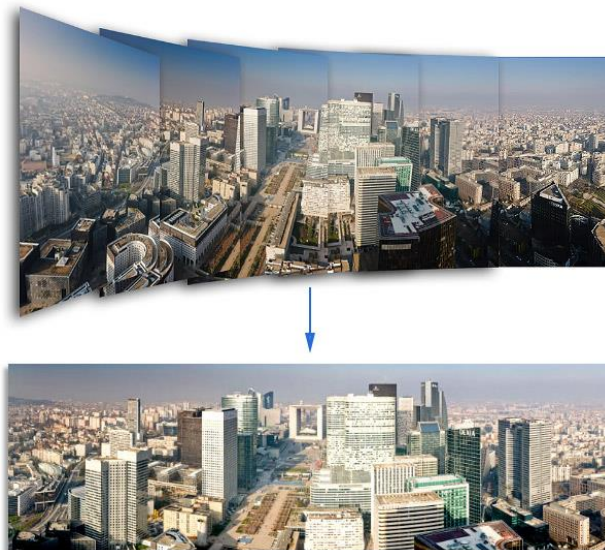
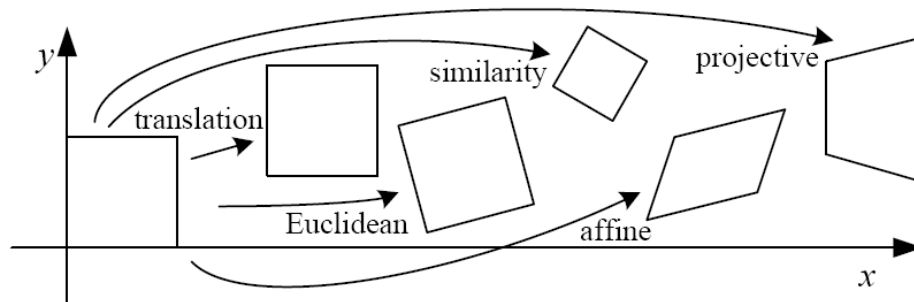
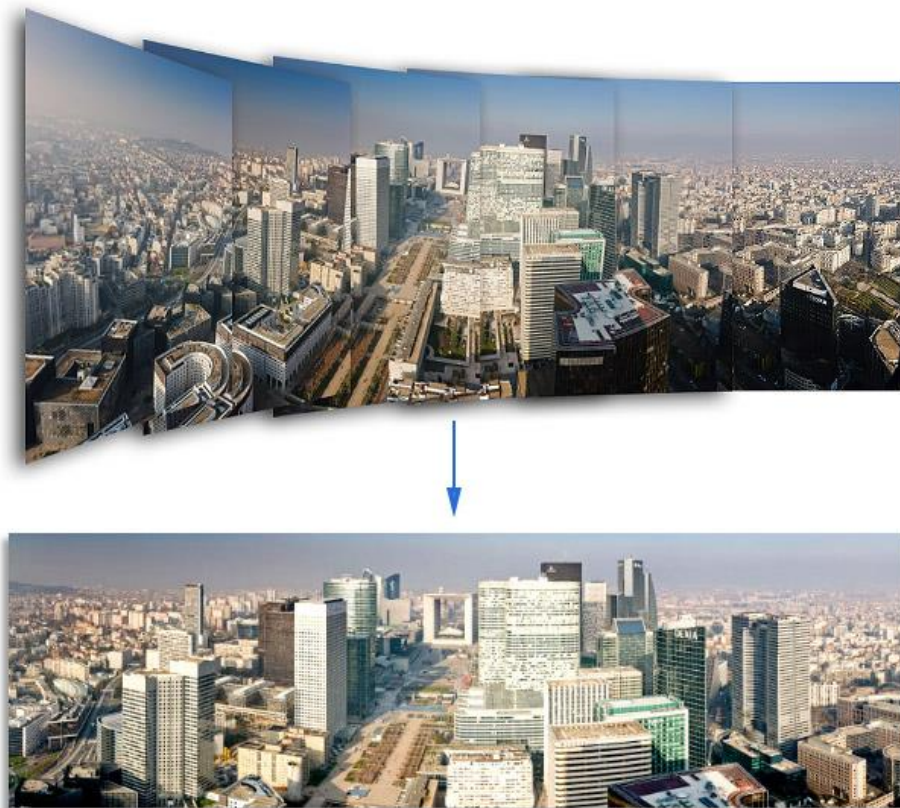


Image Stitching



Goal: Combine two or more overlapping images into one large image.

Typical Steps:

- Transformation computation
 - Reprojection model
 - Key point matching
 - Solving transformation
 - RANSAC
- Planar mapping
- Blending

Today's Agenda

- **Global operations**

- Global transformations
- Image Warping
- Image Morphing

- **Image Stitching**

- Transformation computation
 - Reprojection model
 - Key point matching
 - RANSAC
 - Solving transformation
- Planar mapping
- Blending
 - Feathering
 - Multi-band blending
 - Further improvements

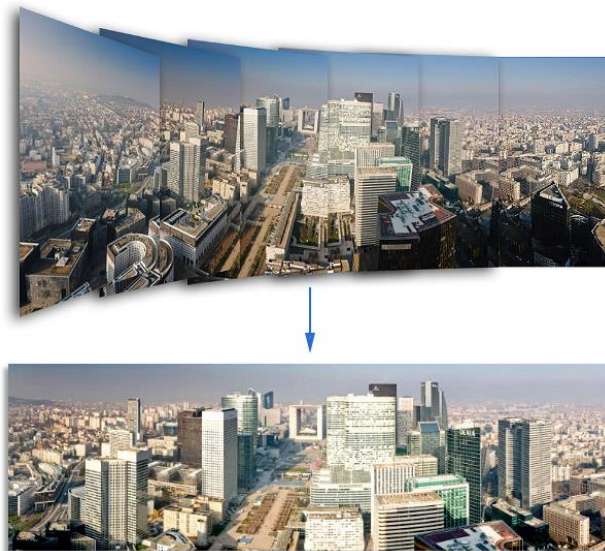
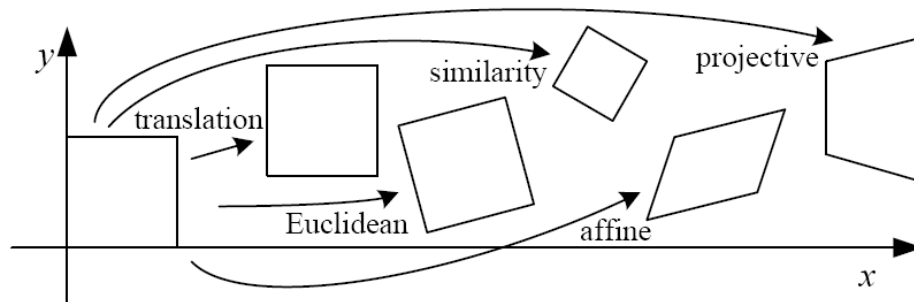
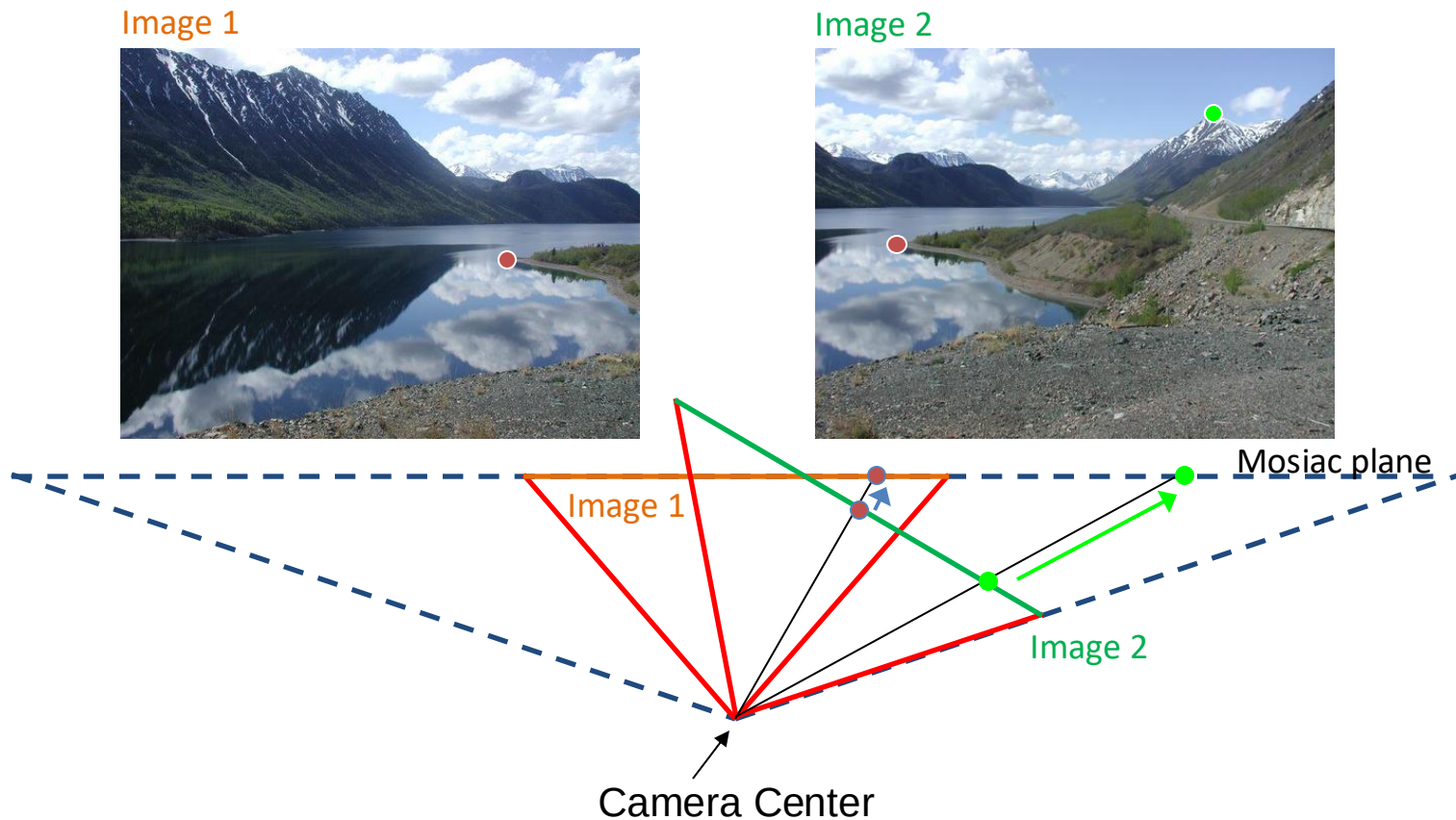


Image Reprojection Model



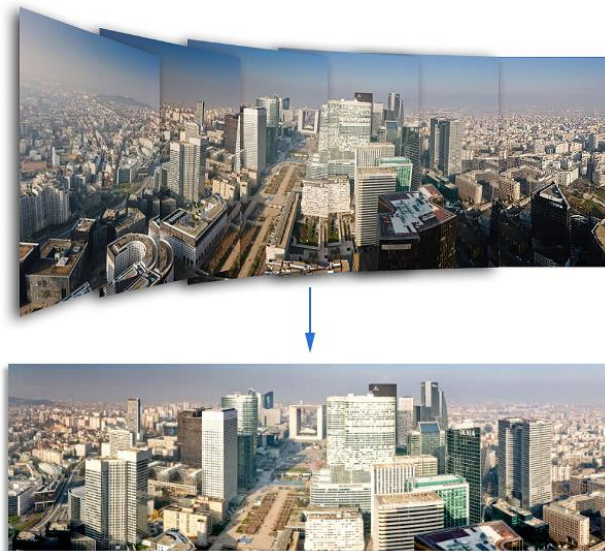
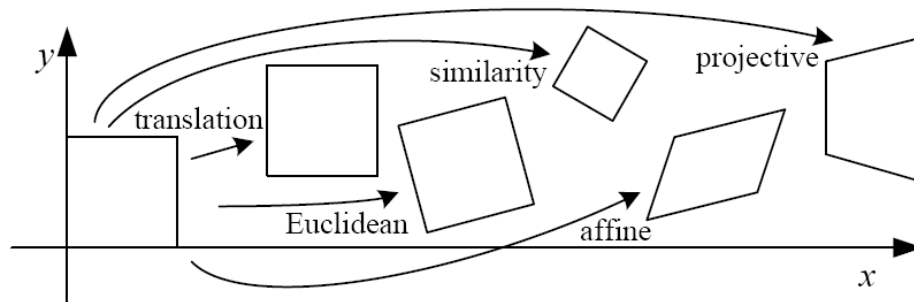
Today's Agenda

- **Global operations**

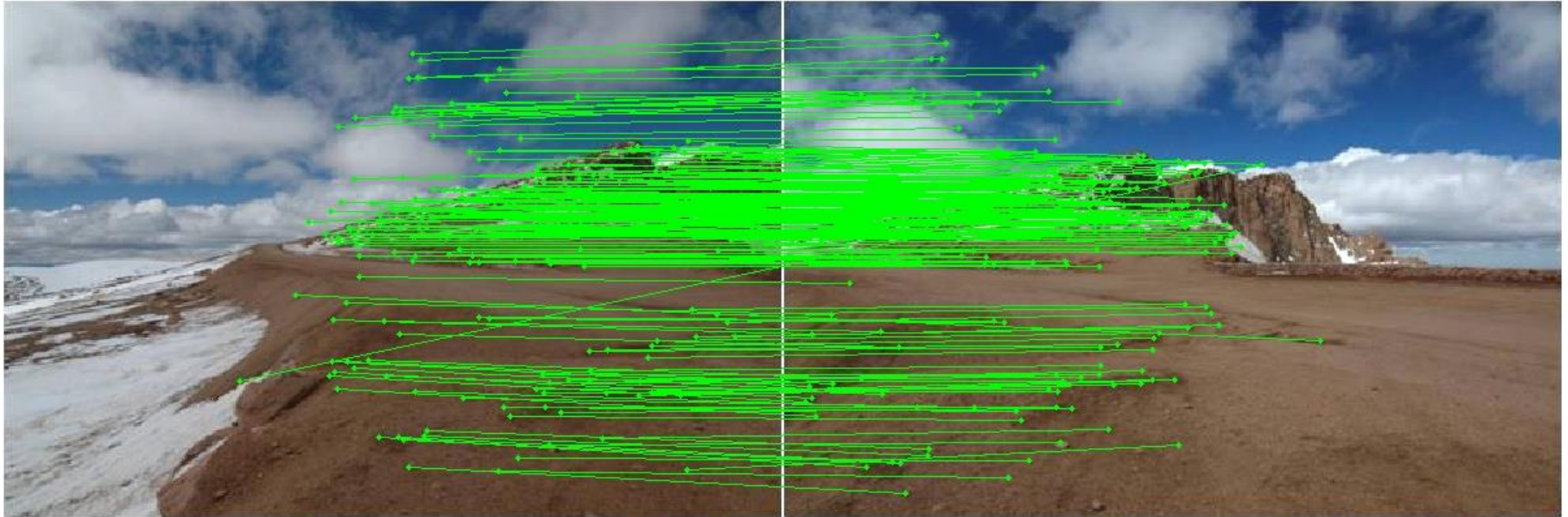
- Global transformations
- Image Warping
- Image Morphing

- **Image Stitching**

- Transformation computation
 - Reprojection model
 - Key point matching
 - RANSAC
 - Solving transformation
- Planar mapping
- Blending
 - Feathering
 - Multi-band blending
 - Further improvements



Keypoint Matching



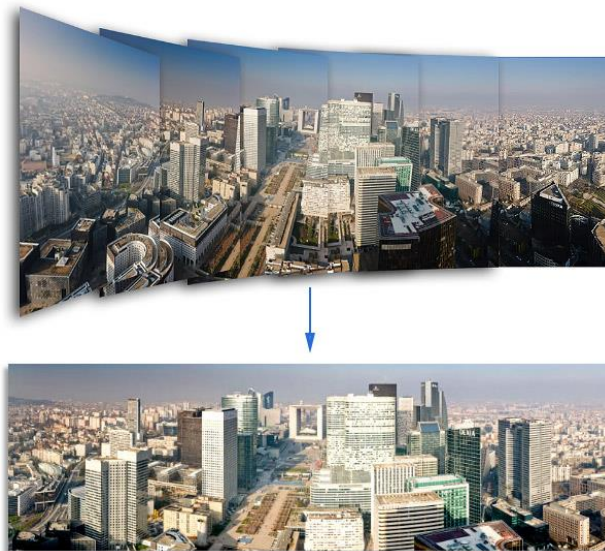
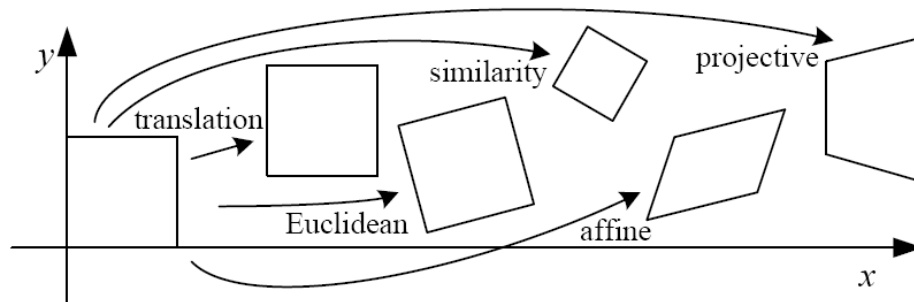
Today's Agenda

- **Global operations**

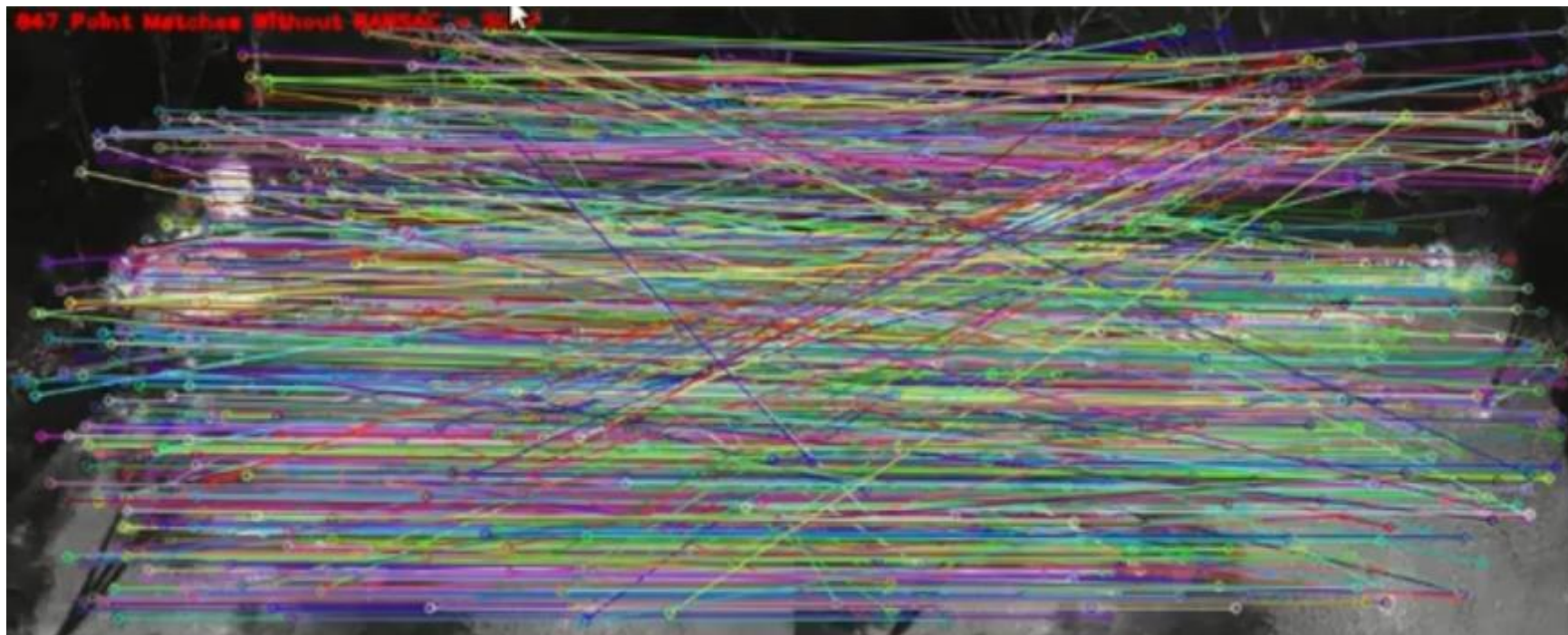
- Global transformations
- Image Warping
- Image Morphing

- **Image Stitching**

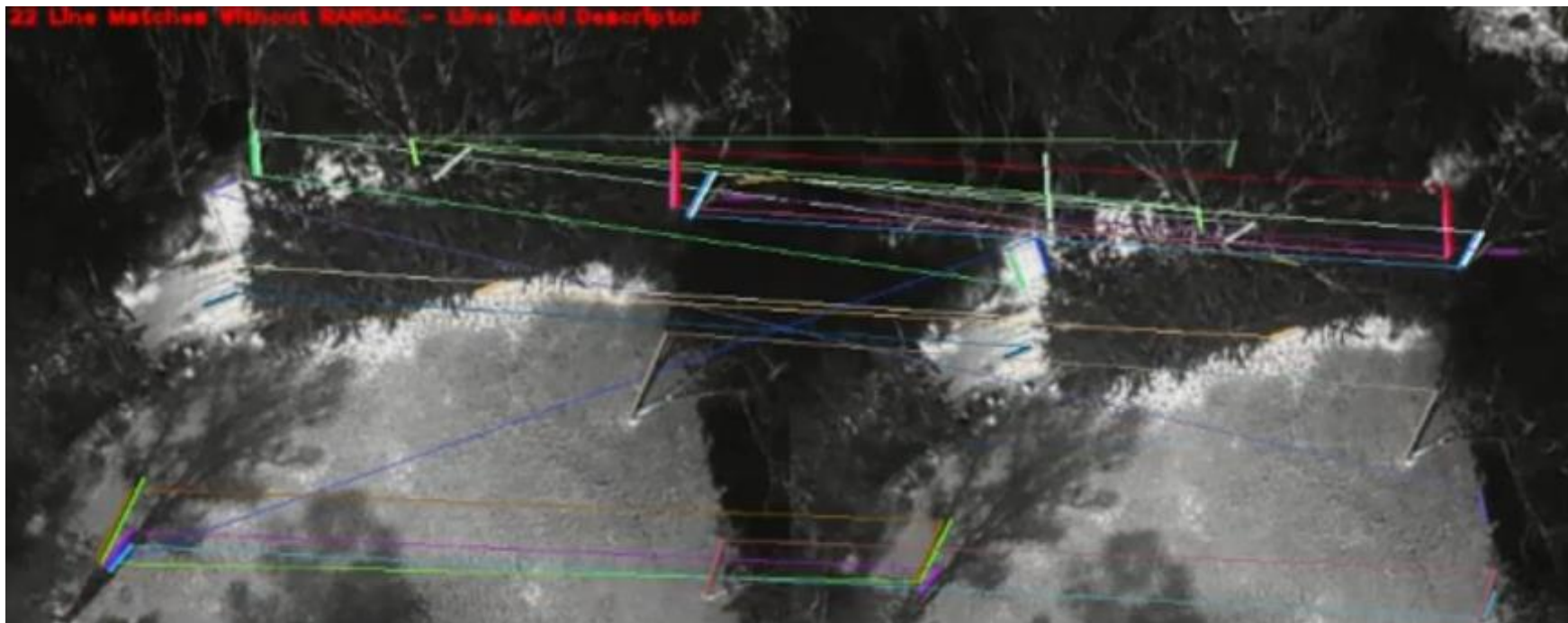
- Transformation computation
 - Reprojection model
 - Key point matching
 - RANSAC
 - Solving transformation
- Planar mapping
- Blending
 - Feathering
 - Multi-band blending
 - Further improvements



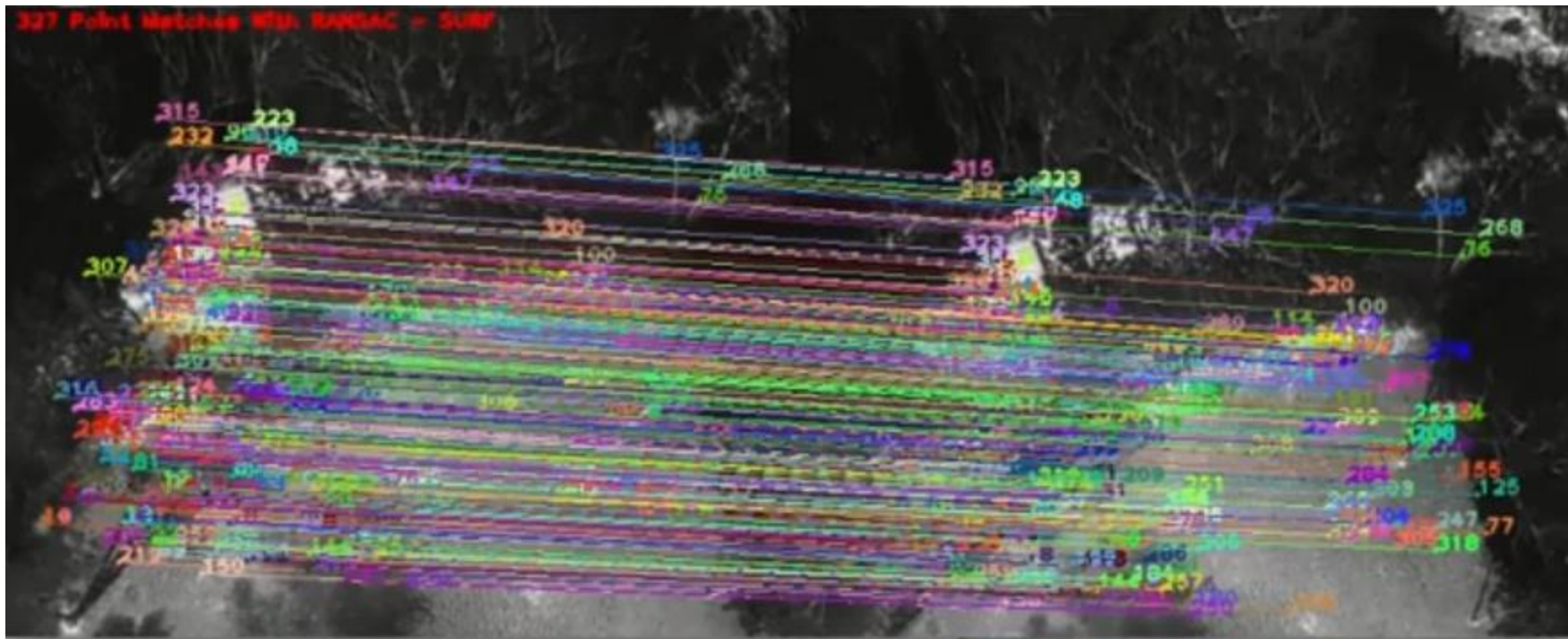
Correspondence Estimation: Matching Pool



Random select and compute transform



Check how many matches follow the transform



Today's Agenda

- **Global operations**

- Global transformations
- Image Warping
- Image Morphing

- **Image Stitching**

- Transformation computation
 - Reprojection model
 - Key point matching
 - Solving transformation
 - RANSAC
- Planar mapping
- Blending
 - Feathering
 - Multi-band blending
 - Further improvements

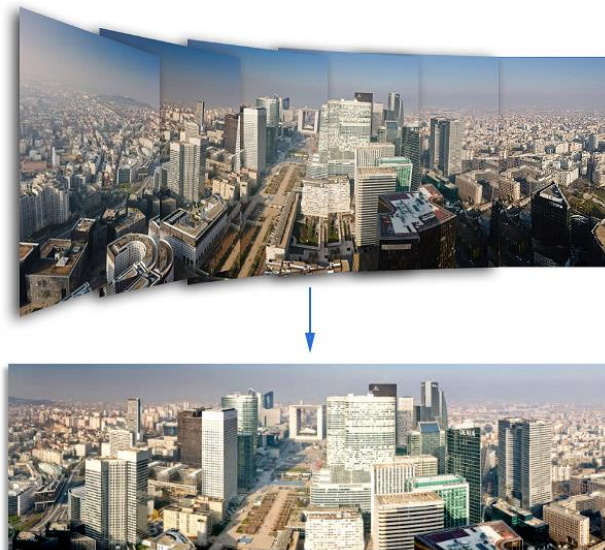
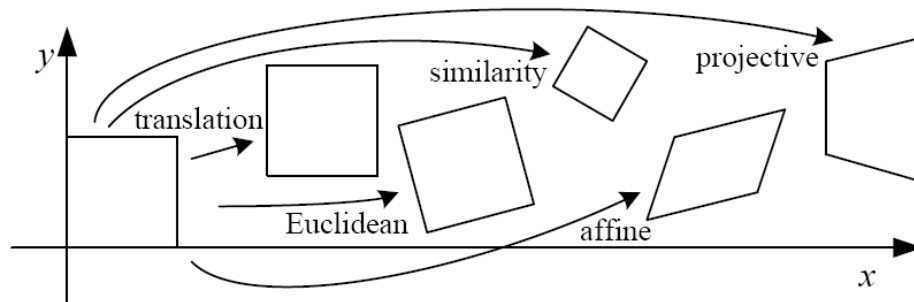
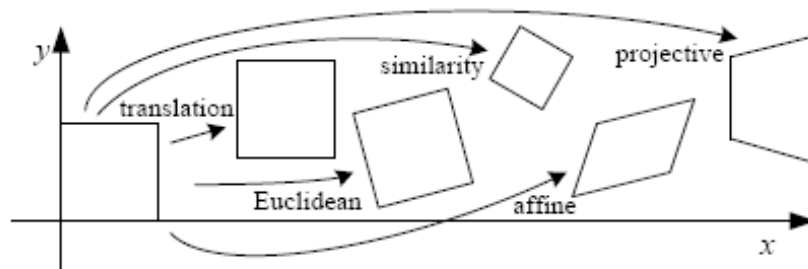


Image Transformation Models

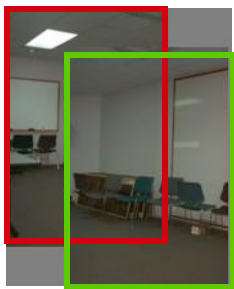


2D Translation

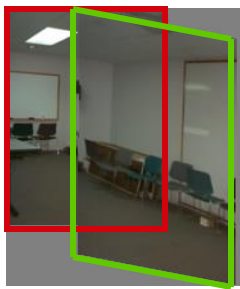
2D Affine

3D Perspective

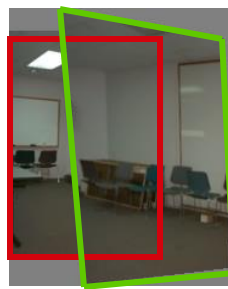
3D rotation



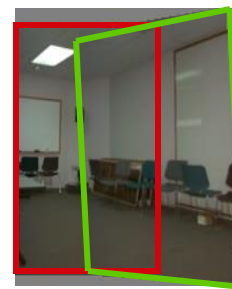
2 unknowns



6 unknowns

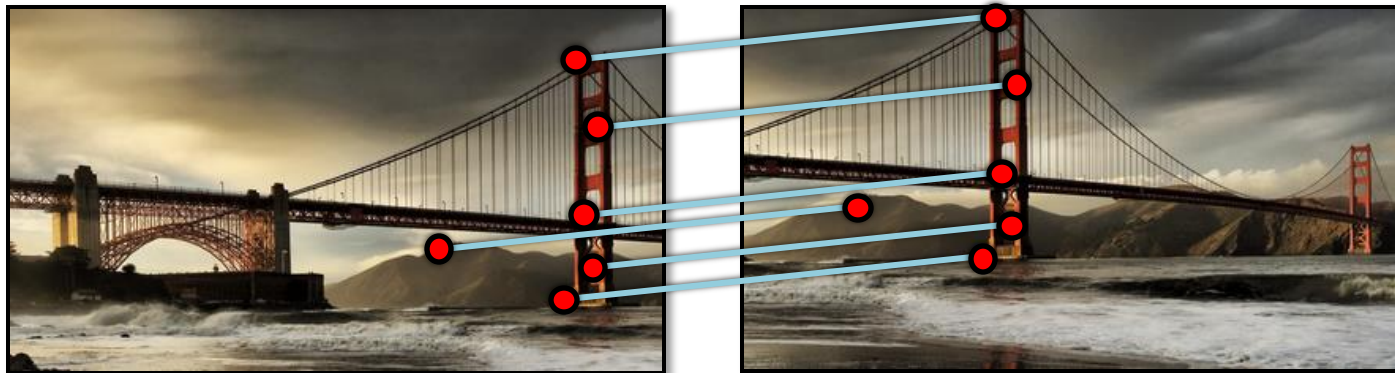


8 unknowns



3 unknowns

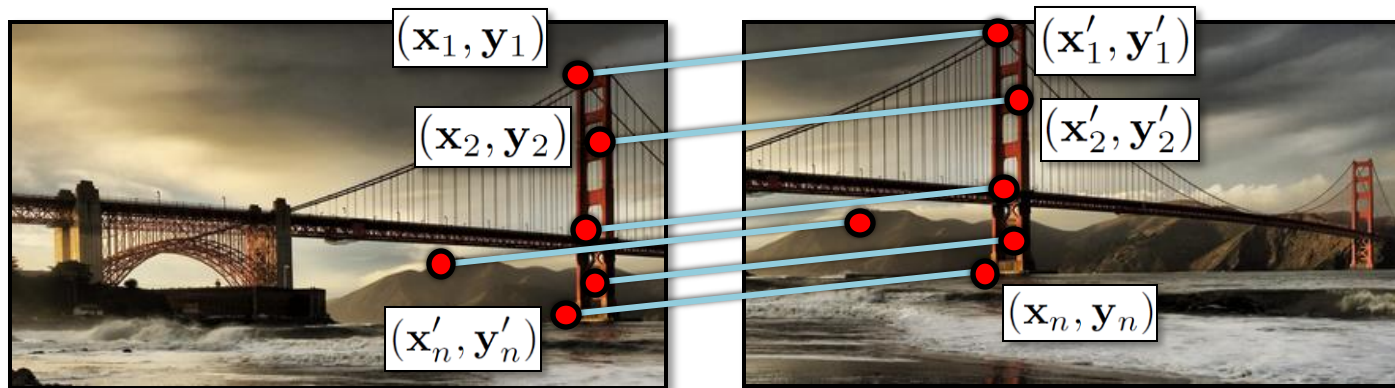
Simple Case: (I) Translation



(x_t, y_t)

How do we solve for
 (x_t, y_t) ?

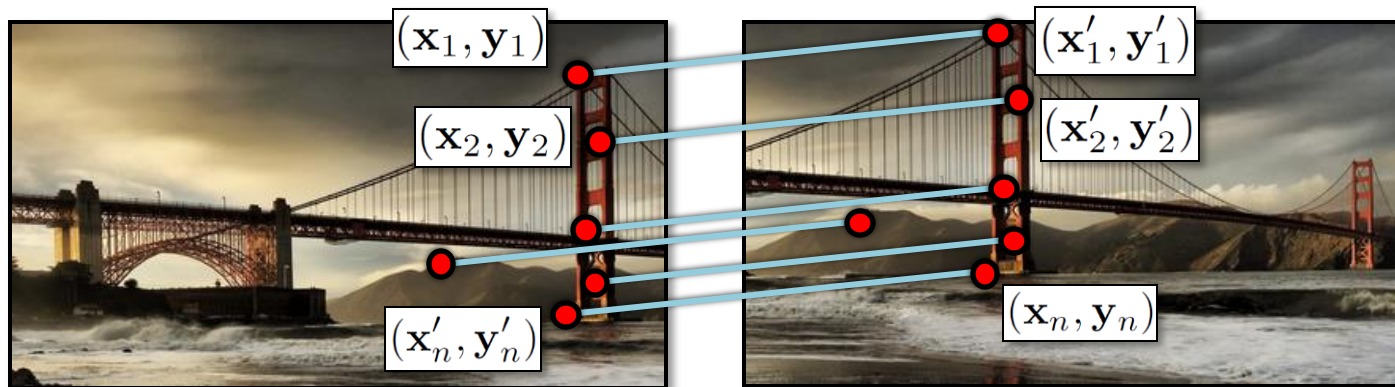
Simple Case: (I) Translation



Displacement of match $i = (\mathbf{x}'_i - \mathbf{x}_i, \mathbf{y}'_i - \mathbf{y}_i)$

Mean displacement $(\mathbf{x}_t, \mathbf{y}_t) = \left(\frac{1}{n} \sum_{i=1}^n \mathbf{x}'_i - \mathbf{x}_i, \frac{1}{n} \sum_{i=1}^n \mathbf{y}'_i - \mathbf{y}_i \right)$

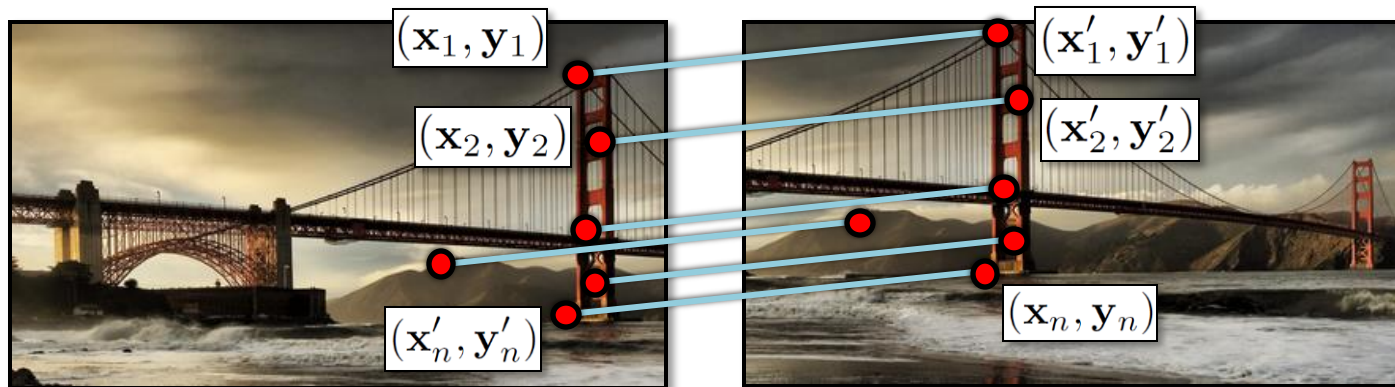
Simple Case: (I) Translation



$$\mathbf{x}_i + \mathbf{x}_t = \mathbf{x}'_i$$
$$\mathbf{y}_i + \mathbf{y}_t = \mathbf{y}'_i$$

- System of linear equations
 - What are the knowns? Unknowns?
 - How many unknowns? How many equations (per match)?

Simple Case: (I) Translation



$$\begin{aligned} \mathbf{x}_i + \mathbf{x}_t &= \mathbf{x}'_i \\ \mathbf{y}_i + \mathbf{y}_t &= \mathbf{y}'_i \end{aligned}$$

- Problem: more equations than unknowns
 - “Overdetermined” system of equations
 - We will find the *least squares* solution

Least squares formulation

- For each point $(\mathbf{x}_i, \mathbf{y}_i)$

$$\mathbf{x}_i + \mathbf{x}_t = \mathbf{x}'_i$$

$$\mathbf{y}_i + \mathbf{y}_t = \mathbf{y}'_i$$

- we define the *residuals* as

$$r_{\mathbf{x}_i}(\mathbf{x}_t) = (\mathbf{x}_i + \mathbf{x}_t) - \mathbf{x}'_i$$

$$r_{\mathbf{y}_i}(\mathbf{y}_t) = (\mathbf{y}_i + \mathbf{y}_t) - \mathbf{y}'_i$$

Least squares formulation

- Goal: minimize sum of squared residuals

$$C(\mathbf{x}_t, \mathbf{y}_t) = \sum_{i=1}^n \left(r_{\mathbf{x}_i}(\mathbf{x}_t)^2 + r_{\mathbf{y}_i}(\mathbf{y}_t)^2 \right)$$

- “Least squares” solution
- For translations, is equal to mean displacement

Least squares

$$\mathbf{A}\mathbf{t} = \mathbf{b}$$

- Find $\mathbf{t}$ that minimizes

$$||\mathbf{A}\mathbf{t} - \mathbf{b}||^2$$

- To solve, form the *normal equations*

$$\mathbf{A}^T \mathbf{A} \mathbf{t} = \mathbf{A}^T \mathbf{b}$$

$$\mathbf{t} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{b}$$

Solving for (I) Translations

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \\ \vdots & \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_t \\ y_t \end{bmatrix} = \begin{bmatrix} x'_1 - x_1 \\ y'_1 - y_1 \\ x'_2 - x_2 \\ y'_2 - y_2 \\ \vdots \\ x'_n - x_n \\ y'_n - y_n \end{bmatrix}$$

$$\begin{matrix} \mathbf{A} \\ 2n \times 2 \end{matrix} \begin{matrix} \mathbf{t} \\ 2 \times 1 \end{matrix} = \begin{matrix} \mathbf{b} \\ 2n \times 1 \end{matrix}$$

Solving for (II) Affine Transformations

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- How many unknowns?
- How many equations per match?
- How many matches do we need?

Solving for (II) Affine Transformations

- Residuals:

$$r_{x_i}(a, b, c, d, e, f) = (ax_i + by_i + c) - x'_i$$

$$r_{y_i}(a, b, c, d, e, f) = (dx_i + ey_i + f) - y'_i$$

- Cost function:

$$C(a, b, c, d, e, f) = \sum_{i=1}^n (r_{x_i}(a, b, c, d, e, f)^2 + r_{y_i}(a, b, c, d, e, f)^2)$$

Solving for (II) Affine Transformations

- Matrix form

$$\begin{bmatrix}
 x_1 & y_1 & 1 & 0 & 0 & 0 \\
 0 & 0 & 0 & x_1 & y_1 & 1 \\
 x_2 & y_2 & 1 & 0 & 0 & 0 \\
 0 & 0 & 0 & x_2 & y_2 & 1 \\
 \vdots & & & & & \\
 x_n & y_n & 1 & 0 & 0 & 0 \\
 0 & 0 & 0 & x_n & y_n & 1
 \end{bmatrix}
 \begin{bmatrix}
 a \\
 b \\
 c \\
 d \\
 e \\
 f
 \end{bmatrix}
 =
 \begin{bmatrix}
 x'_1 \\
 y'_1 \\
 x'_2 \\
 y'_2 \\
 \vdots \\
 x'_n \\
 y'_n
 \end{bmatrix}$$

$$\underset{2n \times 6}{\mathbf{A}} \underset{6 \times 1}{\mathbf{t}} = \underset{2n \times 1}{\mathbf{b}}$$

$$\begin{bmatrix} x'_i \\ y'_i \\ 1 \end{bmatrix} \cong \begin{bmatrix} h_{00} & h_{01} & h_{02} \\ h_{10} & h_{11} & h_{12} \\ h_{20} & h_{21} & h_{22} \end{bmatrix} \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix}$$

$$x'_i = \frac{h_{00}x_i + h_{01}y_i + h_{02}}{h_{20}x_i + h_{21}y_i + h_{22}}$$

$$y'_i = \frac{h_{10}x_i + h_{11}y_i + h_{12}}{h_{20}x_i + h_{21}y_i + h_{22}}$$

$$x'_i(h_{20}x_i + h_{21}y_i + h_{22}) = h_{00}x_i + h_{01}y_i + h_{02}$$

$$y'_i(h_{20}x_i + h_{21}y_i + h_{22}) = h_{10}x_i + h_{11}y_i + h_{12}$$

Solving for (III) Homographies

Beyond CV1



$$x'_i(h_{20}x_i + h_{21}y_i + h_{22}) = h_{00}x_i + h_{01}y_i + h_{02}$$

$$y'_i(h_{20}x_i + h_{21}y_i + h_{22}) = h_{10}x_i + h_{11}y_i + h_{12}$$

$$\begin{bmatrix} x_i & y_i & 1 & 0 & 0 & 0 & -x'_i x_i & -x'_i y_i & -x'_i \\ 0 & 0 & 0 & x_i & y_i & 1 & -y'_i x_i & -y'_i y_i & -y'_i \end{bmatrix} \begin{bmatrix} h_{00} \\ h_{01} \\ h_{02} \\ h_{10} \\ h_{11} \\ h_{12} \\ h_{20} \\ h_{21} \\ h_{22} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix}
 x_1 & y_1 & 1 & 0 & 0 & 0 & -x'_1 x_1 & -x'_1 y_1 & -x'_1 \\
 0 & 0 & 0 & x_1 & y_1 & 1 & -y'_1 x_1 & -y'_1 y_1 & -y'_1 \\
 & & & & & \vdots & & & \\
 x_n & y_n & 1 & 0 & 0 & 0 & -x'_n x_n & -x'_n y_n & -x'_n \\
 0 & 0 & 0 & x_n & y_n & 1 & -y'_n x_n & -y'_n y_n & -y'_n
 \end{bmatrix}
 \begin{bmatrix}
 h_{00} \\
 h_{01} \\
 h_{02} \\
 h_{10} \\
 h_{11} \\
 h_{12} \\
 h_{20} \\
 h_{21} \\
 h_{22}
 \end{bmatrix}
 =
 \begin{bmatrix}
 0 \\
 0 \\
 \vdots \\
 0 \\
 0
 \end{bmatrix}$$

$\mathbf{A}$
 $2n \times 9$

$\mathbf{h}$
 9×1

$\mathbf{0}$
 $2n \times 1$

Defines a least squares problem:

minimize $\|\mathbf{A}\mathbf{h} - \mathbf{0}\|^2$

- Since $\mathbf{h}$ is only defined up to scale, solve for unit vector $\hat{\mathbf{h}}$
- Solution: $\hat{\mathbf{h}}$ = eigenvector of $\mathbf{A}^T \mathbf{A}$ with smallest eigenvalue
- Works with 4 or more points

→ Solution for homography representing the projective camera transformation between two image planes

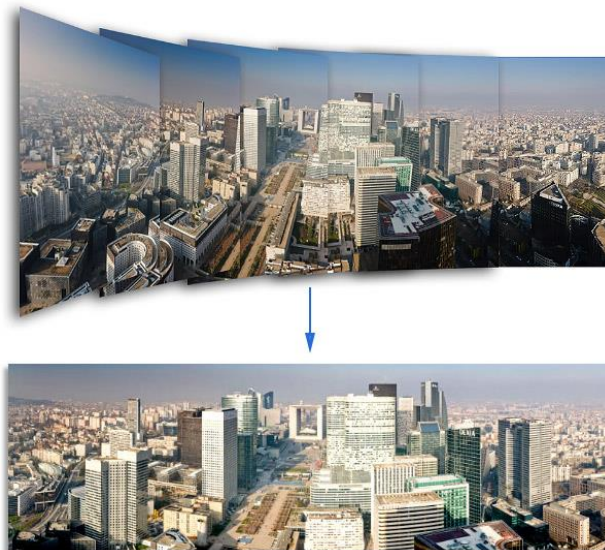
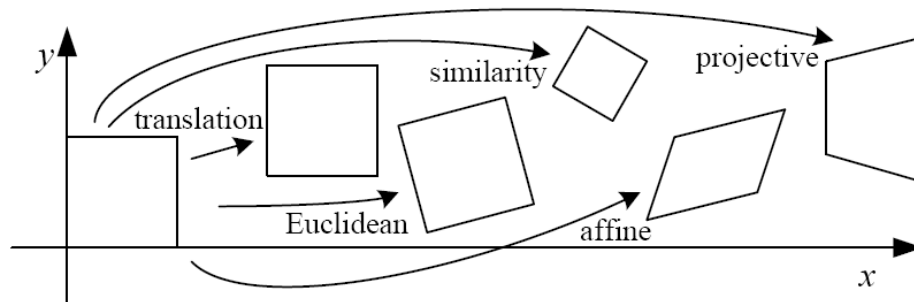
Today's Agenda

- **Global operations**

- Global transformations
- Image Warping
- Image Morphing

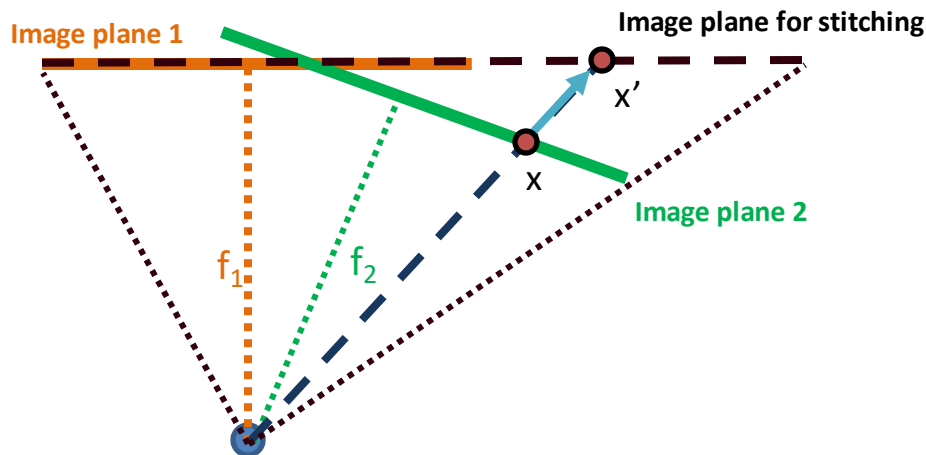
- **Image Stitching**

- Transformation computation
 - Reprojection model
 - Key point matching
 - RANSAC
 - Solving transformation
- Planar mapping
- Blending
 - Feathering
 - Multi-band blending
 - Further improvements

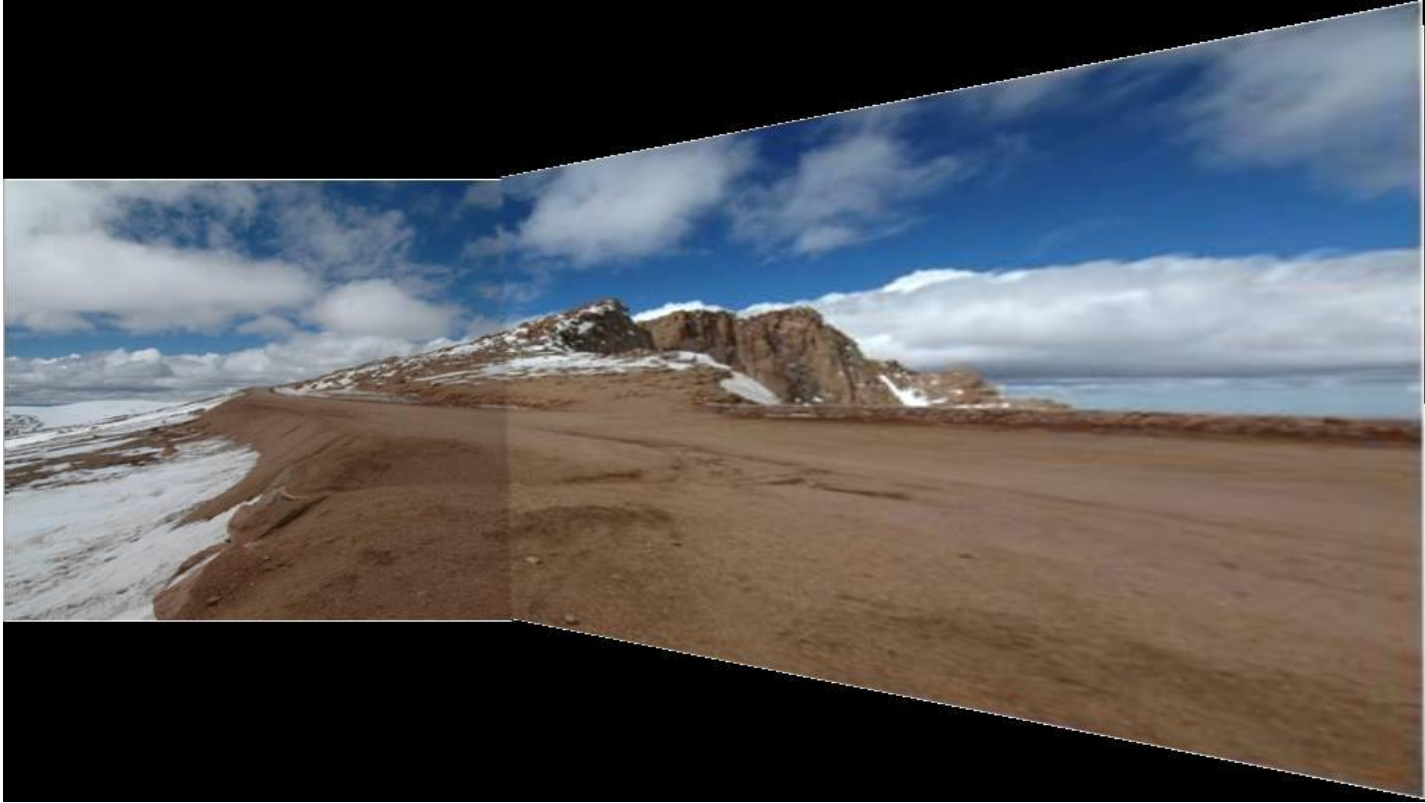


Planar Projection (Homography)

- **Goal:** Reproject several images into a new stitched image.
Here, the image domain is **planar**, but could also be cylindrical, spherical, etc.
- **Approach:** **projective mapping** (homography)
describes how coordinates from one image plane transform to coordinates in another image plane.



Planar Projection (Homography)



[Photo by Russ Hewett]

Planar Projection (Homography)

Planar



[Photo by Russ Hewett]

Today's Agenda

- **Global operations**

- Global transformations
- Image Warping
- Image Morphing

- **Image Stitching**

- Transformation computation
 - Reprojection model
 - Key point matching
 - RANSAC
 - Solving transformation
- Planar mapping
- **Blending**
 - Feathering
 - Multi-band blending
 - Further improvements

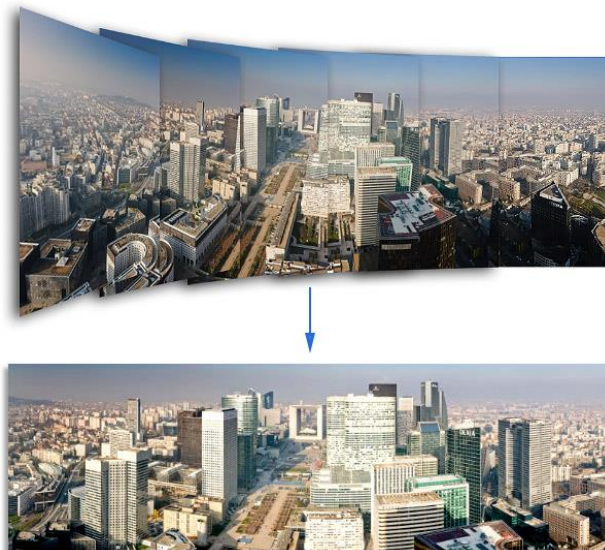
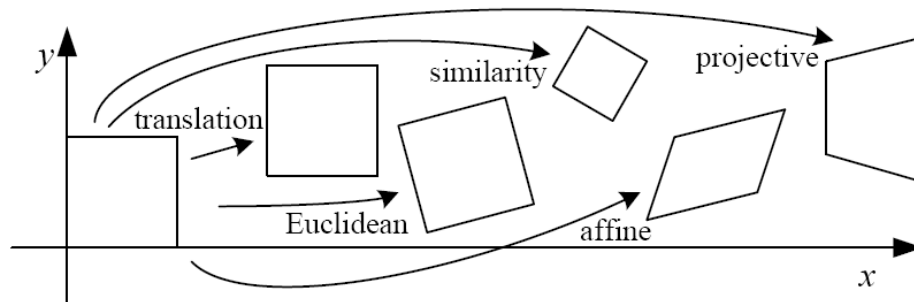
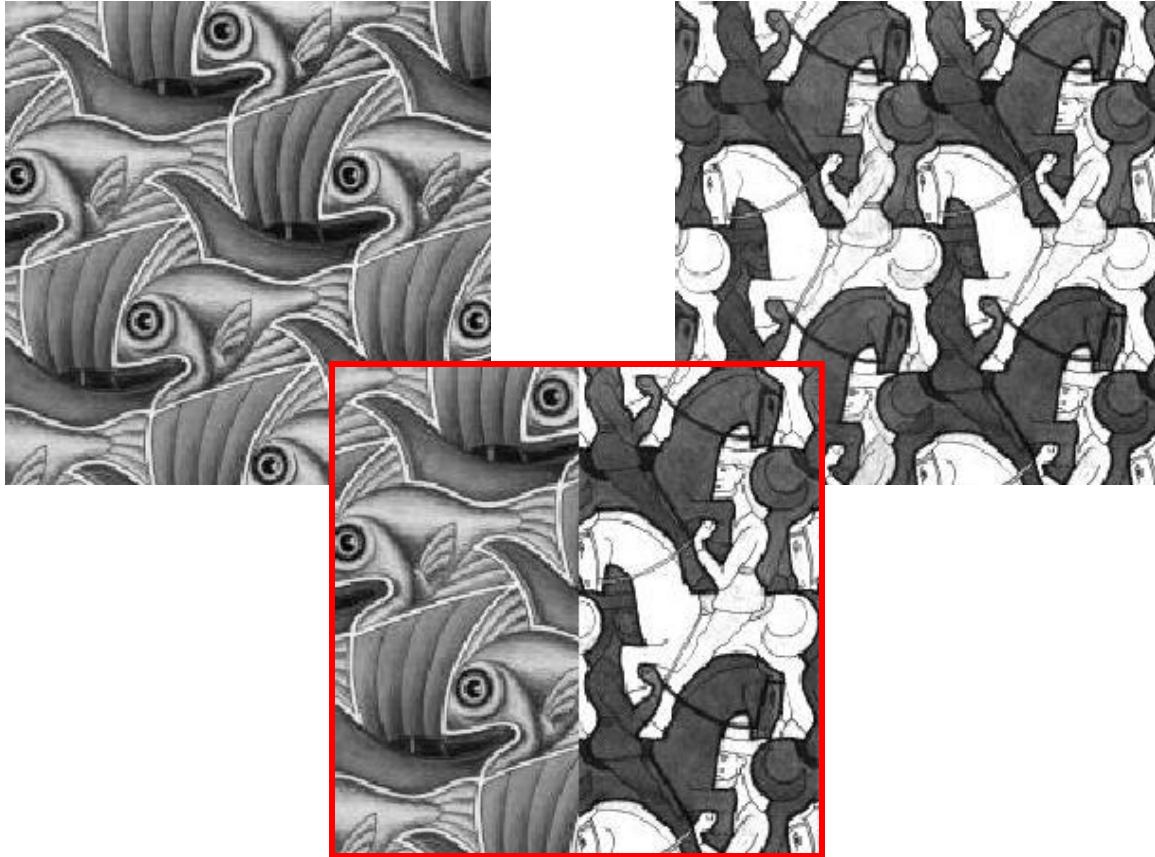


Image Blending



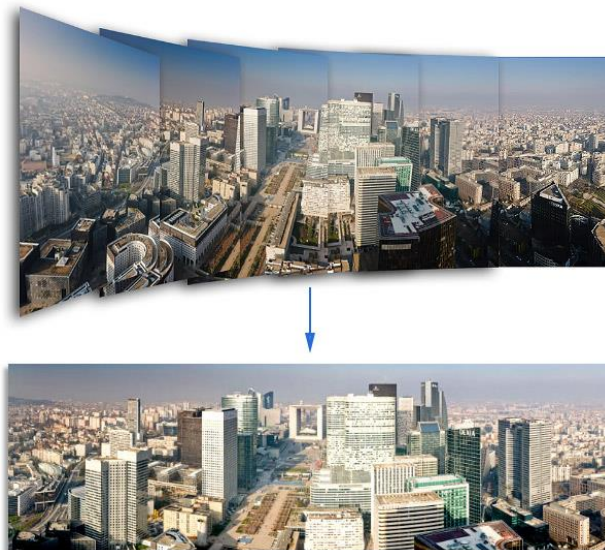
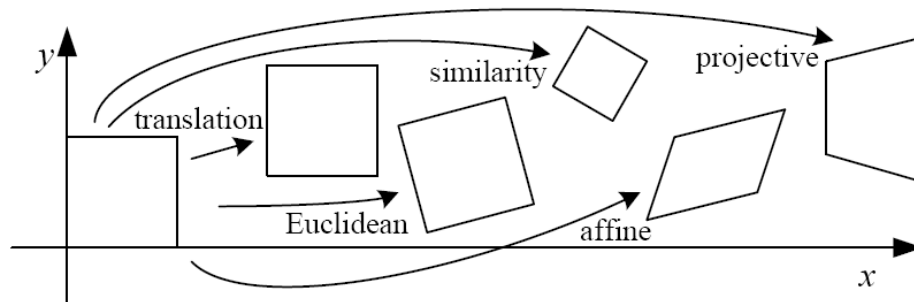
Today's Agenda

- **Global operations**

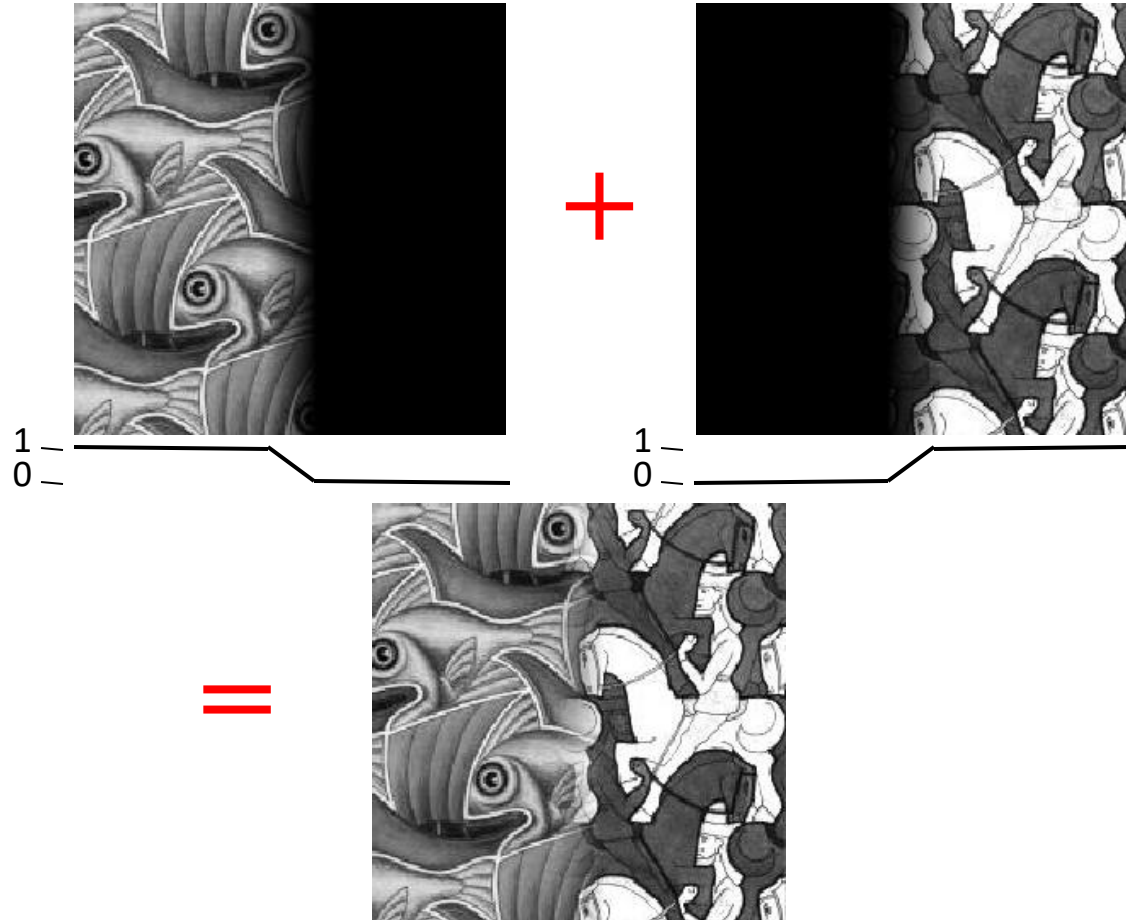
- Global transformations
- Image Warping
- Image Morphing

- **Image Stitching**

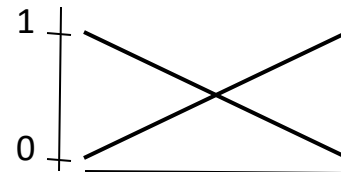
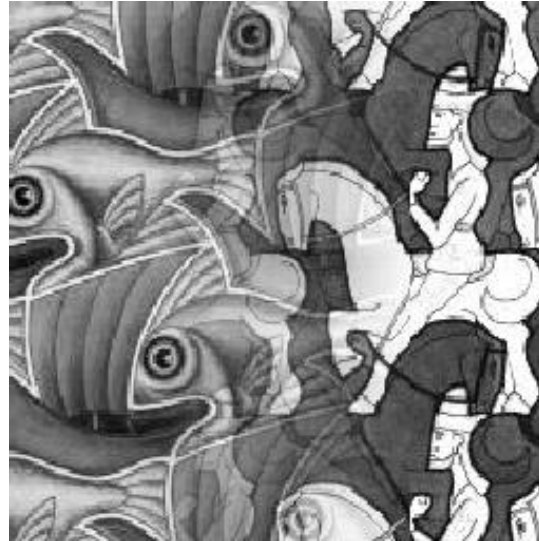
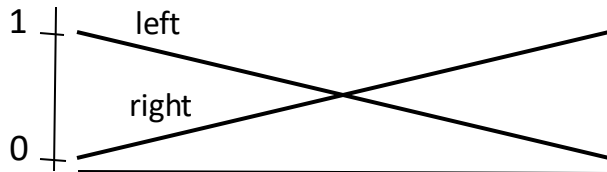
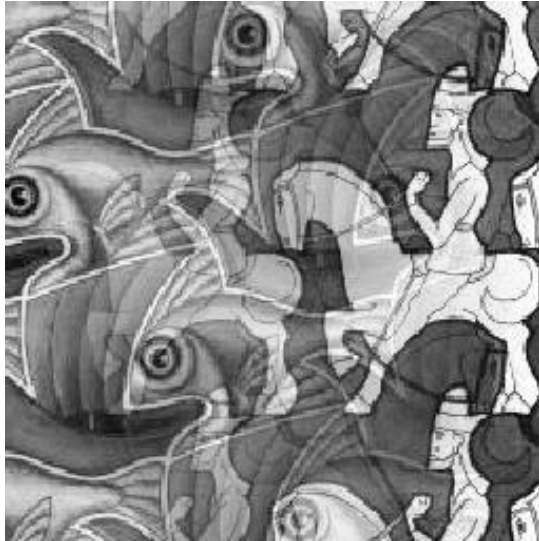
- Transformation computation
 - Reprojection model
 - Key point matching
 - RANSAC
 - Solving transformation
- Planar mapping
- **Blending**
 - **Feathering**
 - Multi-band blending
 - Further improvements



Feathering

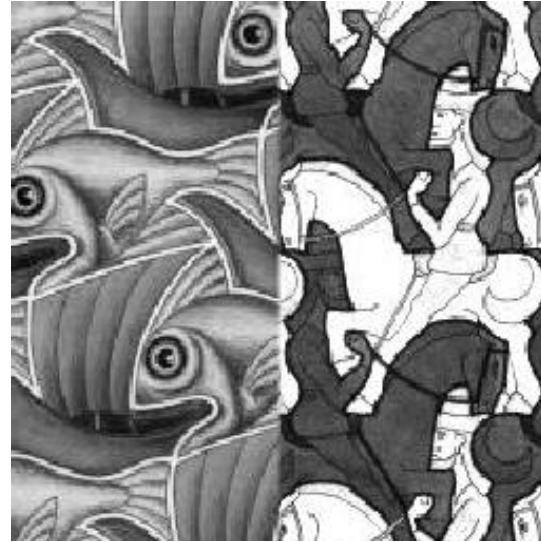
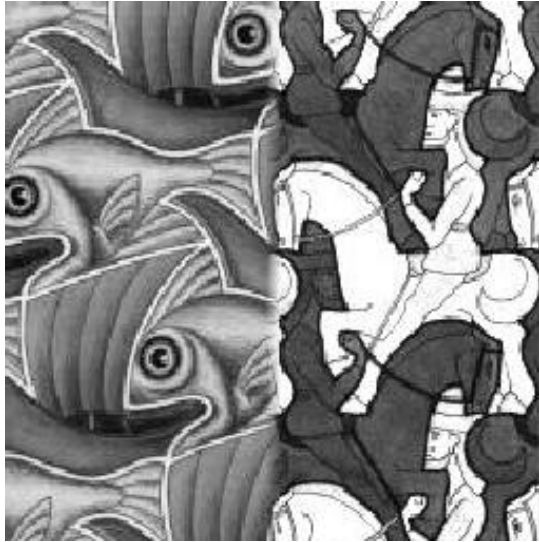


Effect of Window (ramp-width) Size

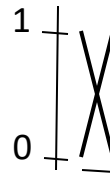


“cross-fading”

Effect of Window (ramp-width) Size



Good Window Size



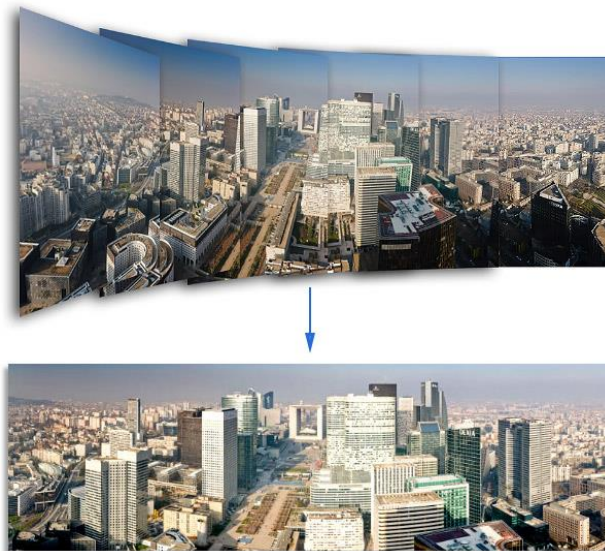
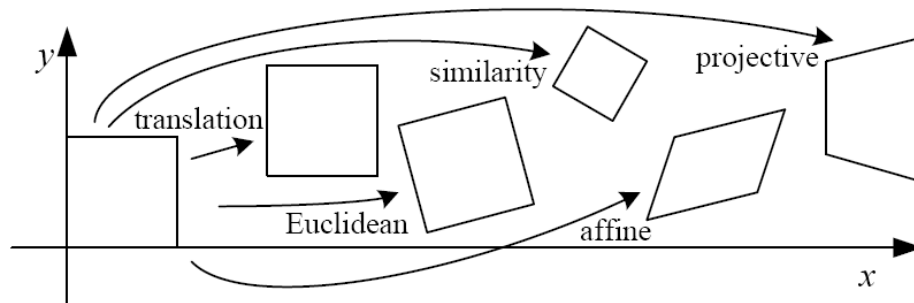
Today's Agenda

- **Global operations**

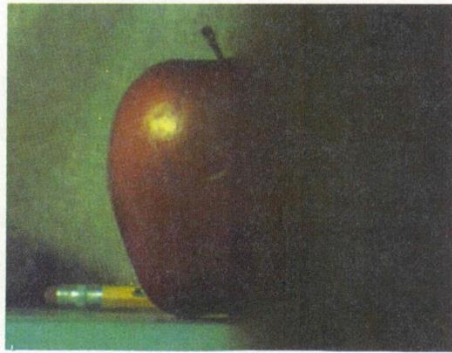
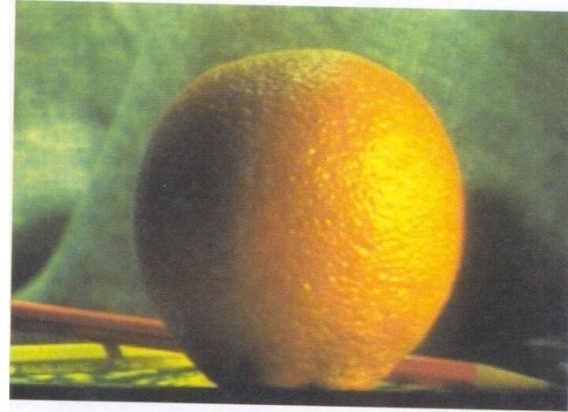
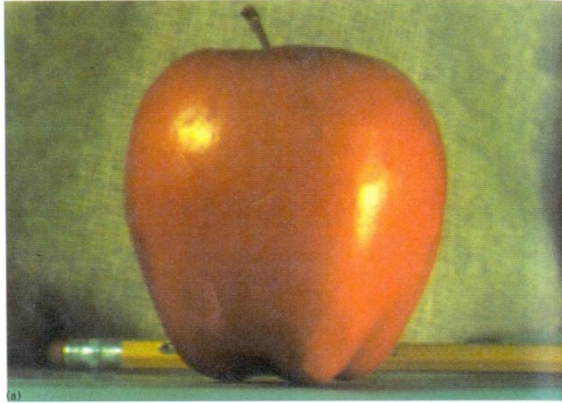
- Global transformations
- Image Warping
- Image Morphing

- **Image Stitching**

- Transformation computation
 - Reprojection model
 - Key point matching
 - RANSAC
 - Solving transformation
- Planar mapping
- **Blending**
 - Feathering
 - **Multi-band blending**
 - Further improvements



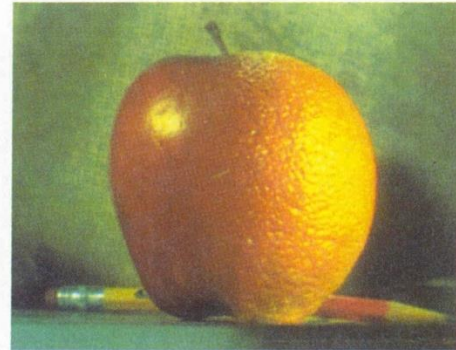
Multi-band Blending



(d)



(h)

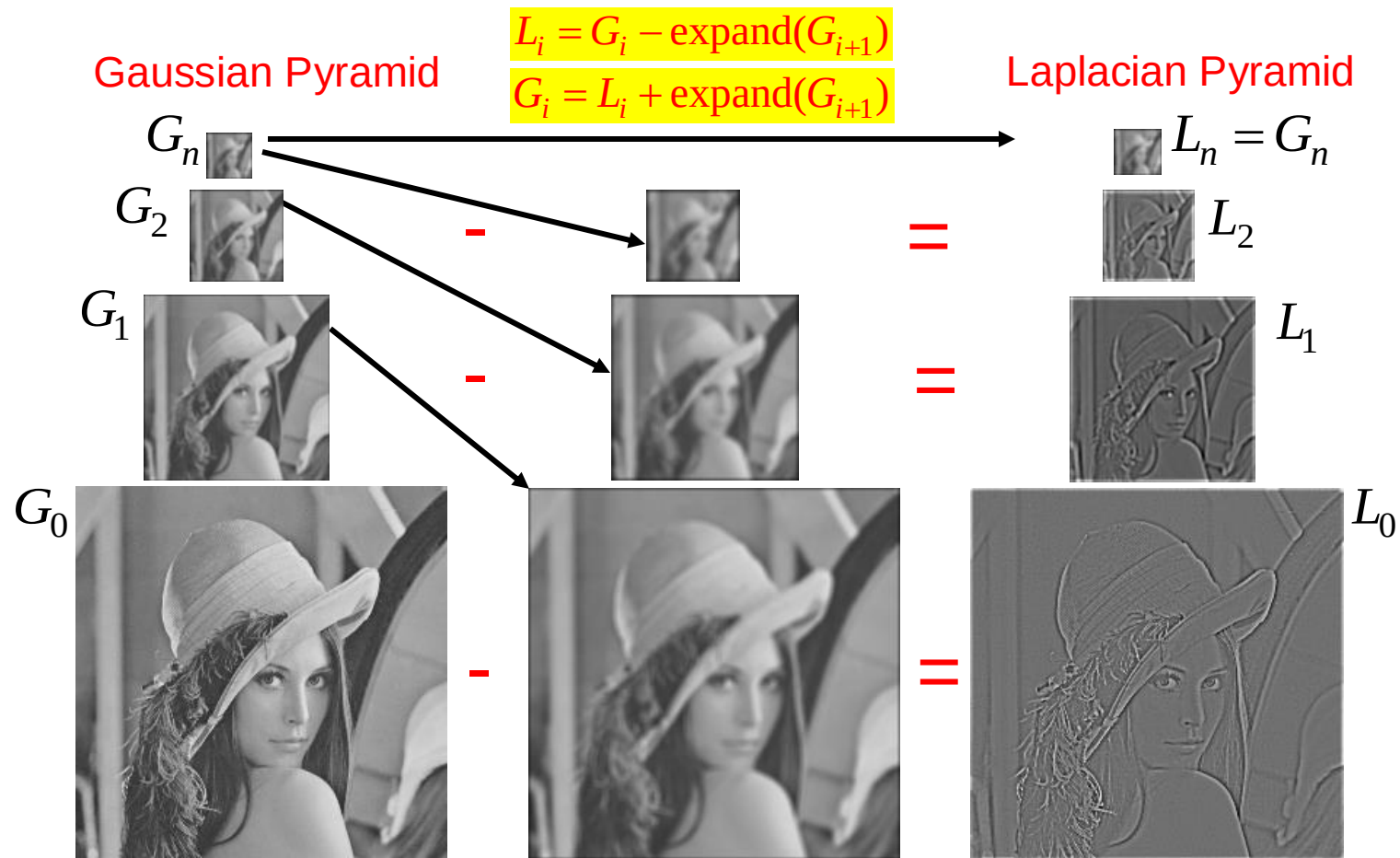


(l)

Create a Laplacian pyramid, blend each level

Burt, P. J. and Adelson, E. H., [A multiresolution spline with applications to image mosaics](#), ACM Transactions on Graphics, 42(4), October 1983, 217-236.

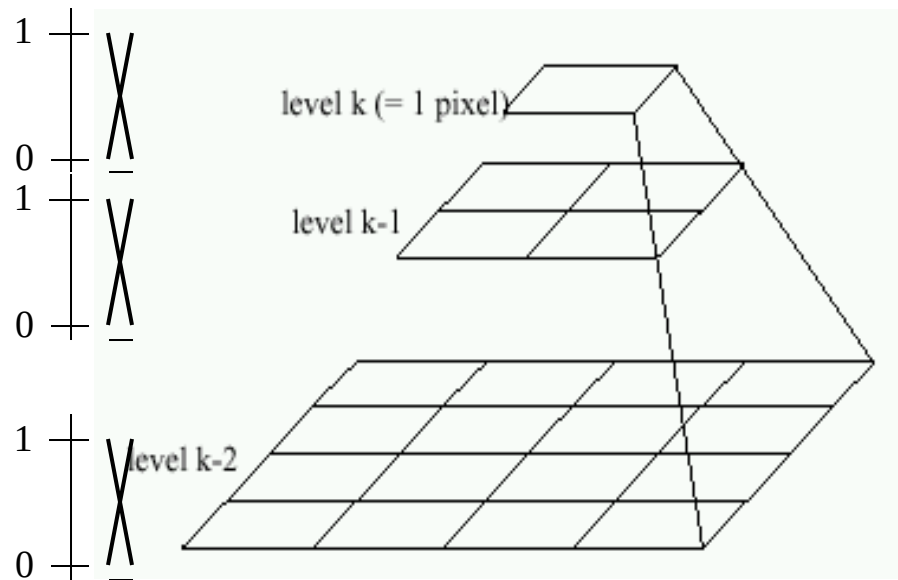
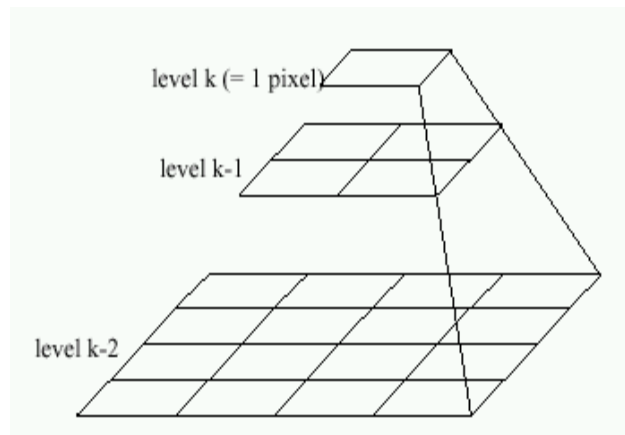
Laplacian Pyramid



Multi-band Blending with Laplacian Pyramid



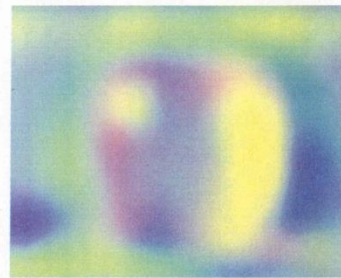
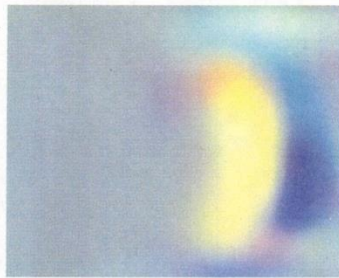
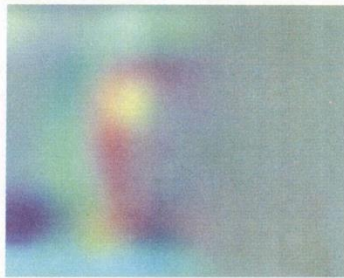
- At low frequencies, blend slowly
- At high frequencies, blend quickly



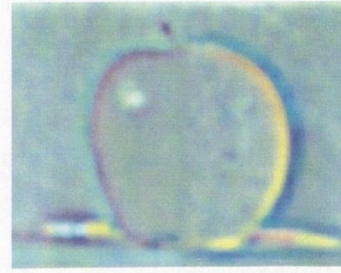
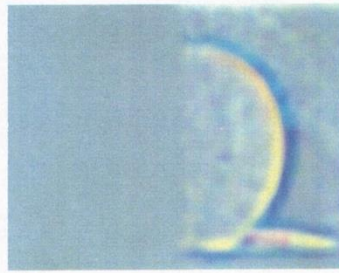
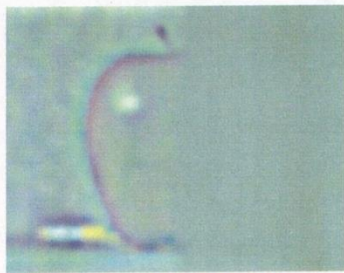
Multi-band Blending with Laplacian Pyramid



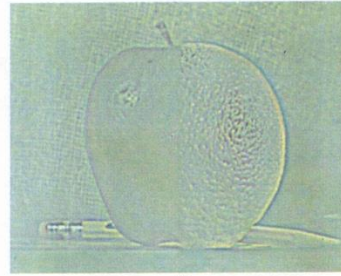
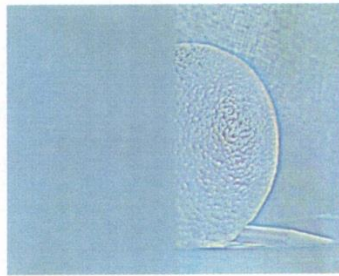
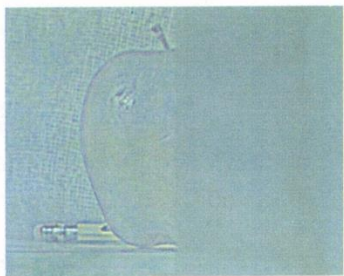
Laplacian
level
4



Laplacian
level
2



Laplacian
level
0



left pyramid

right pyramid

blended pyramid

Multi-band Blending

- Compute Laplacian pyramid of images and mask
- Create blended image at each level of pyramid
 - At low frequencies, blend slowly
 - At high frequencies, blend quickly
- Reconstruct complete image

Multi-band Blending



(a) Linear blending



(b) Multi-band blending

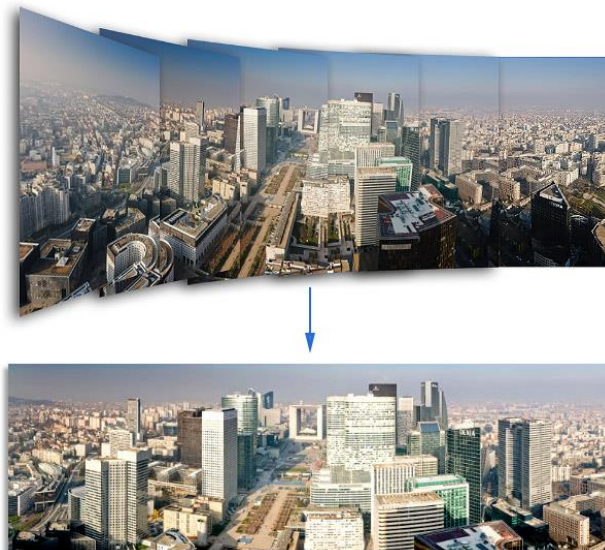
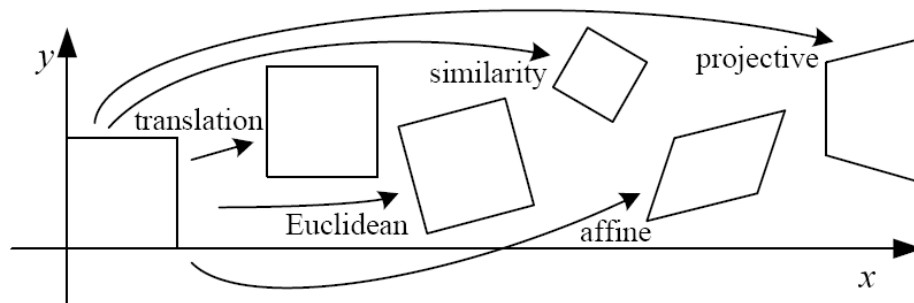
Today's Agenda

- **Global operations**

- Global transformations
- Image Warping
- Image Morphing

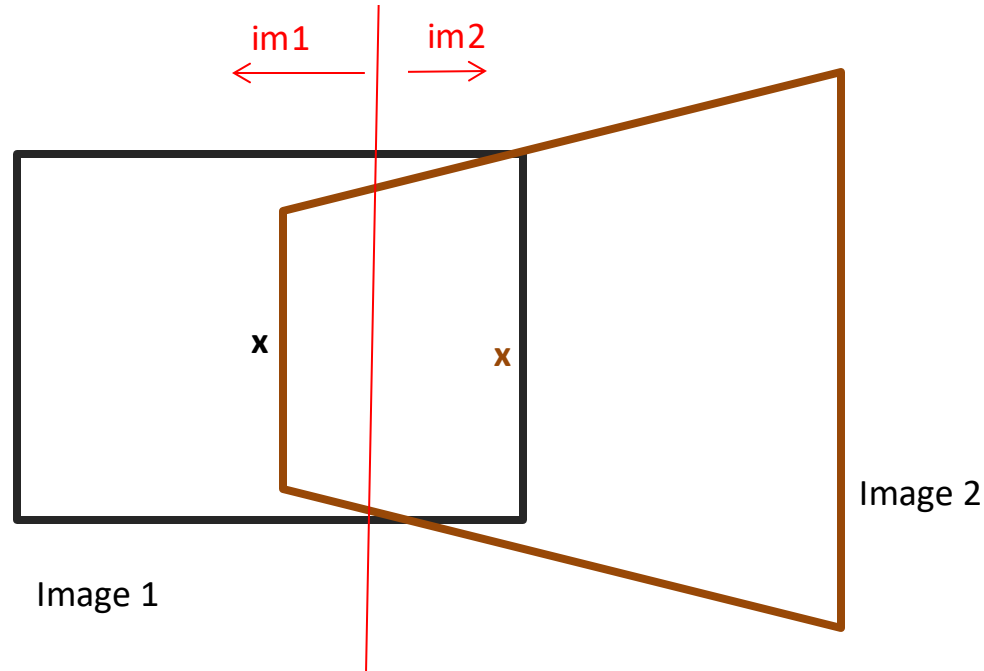
- **Image Stitching**

- Transformation computation
 - Reprojection model
 - Key point matching
 - RANSAC
 - Solving transformation
- Planar mapping
- **Blending**
 - Feathering
 - Multi-band blending
 - Further improvements



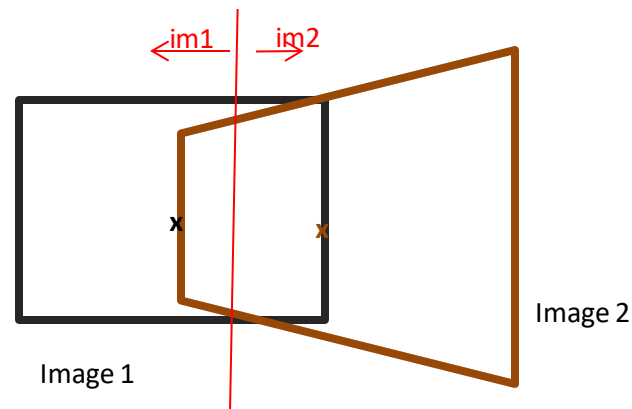
Choosing Seams

- Easy method
 - Assign each pixel to image with nearest center



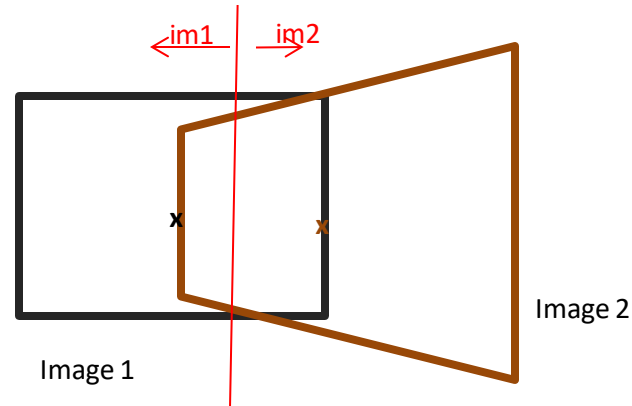
Choosing Seams

- Easy method
 - Assign each pixel to image with nearest center
 - Create a mask:
 - Smooth boundaries (“feathering”):
 - Composite



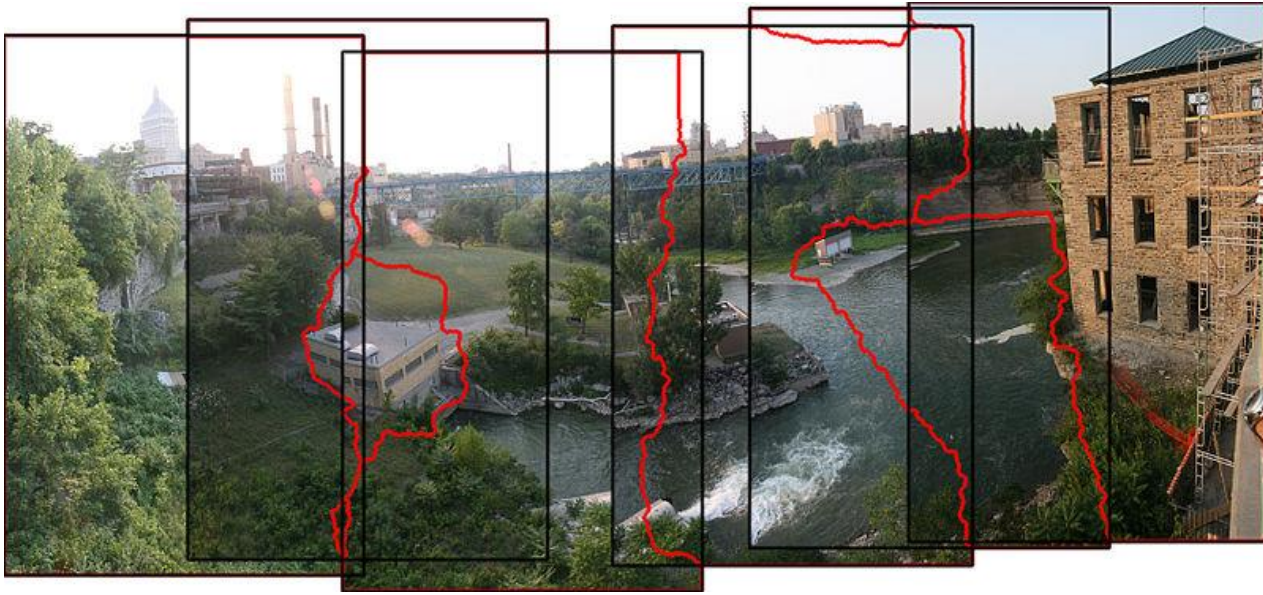
Choosing Seams

- Easy method
 - Assign each pixel to image with nearest center
 - Create a mask:
 - Smooth boundaries (“feathering”):
 - Composite



Choosing Seams

- Better method: dynamic program to find seam along well-matched regions



[Illustration: http://en.wikipedia.org/wiki/File:Rochester_NY.jpg]

Gain Compensation

- Simple gain adjustment
 - Compute average RGB intensity of each image in overlapping region
 - Normalize intensities by ratio of averages



Blending Comparison



(b) Without gain compensation



(c) With gain compensation



(d) With gain compensation and multi-band blending

De-Ghosting

- Remove inconsistencies between pixels from different overlapping images
- Simplest method: use **median** pixel color instead of **average**



Questions?

Disclaimer

Many of the slides used here are obtained from online resources (including many open lecture materials) without appropriate acknowledgement. They are used here for the sole purpose of classroom teaching. All the credit and all the copyrights belong to the original authors. You should not copy it, redistribute it, put it online, or use it for any other purposes than for this course.